PROJET DE FIN D'ANNÉE

2éme année de Cycle d'Ingénierie en Système d'informatique

Sous le thème :

Conception et développement d'une plateforme intelligente de suivi médical "Sihati AI"

Présenté par :

- OUKMI Fatima zahra
- Moujtahid Chaimae
- Lamharzi Hajar

Encadré par :

M. Tabii Younes

Année Universitaire 2025 – 2026

Avant-propos

Ce rapport est le fruit d'un travail collectif réalisé dans le cadre de notre projet de fin d'année, en tant qu'étudiants en développement informatique. Il représente l'aboutissement d'un parcours d'apprentissage riche et formateur, où nous avons pu mobiliser nos compétences techniques, notre sens de l'organisation et notre esprit d'équipe.

Le projet que nous avons développé consiste en une plateforme intelligente de suivi médical, Sihati AI, qui vise à moderniser la communication entre patients et professionnels de santé. En remplaçant les carnets papier, les appels téléphoniques et le suivi manuel des constantes de santé par une solution numérique intuitive et intelligente, nous avons cherché à répondre à un besoin réel tout en respectant des contraintes de sécurité, de fiabilité et de simplicité d'usage.

Ce travail nous a permis de mettre en œuvre les concepts clés de la modélisation UML, de la conception de bases de données, du développement web avec Angular et du développement backend avec Spring Boot, ainsi que l'intégration de fonctionnalités avancées telles que la détection intelligente d'anomalies médicales, un avatar conversationnel vocal, et la reconnaissance optique de caractères (OCR) pour l'analyse des ordonnances.

Ce projet a été également l'occasion de développer nos capacités de collaboration, de répartition des tâches, de communication technique et de résolution de problèmes concrets. Chaque membre du groupe a apporté sa contribution dans un esprit de complémentarité et d'entraide, pour faire aboutir une plateforme complète et fonctionnelle.

Nous avons choisi de prioriser les fonctionnalités à plus forte valeur ajoutée médicale et technologique, afin de recentrer notre travail sur les enjeux réels du suivi de santé numérique. Ce rapport documente notre démarche, nos choix techniques, notre modélisation, ainsi que les principales étapes de développement de la plateforme.

Dédicaces

Nous dédions ce travail, fruit de plusieurs mois d'efforts communs, à nos parents respectifs pour leur amour inconditionnel, leurs encouragements et leurs sacrifices qui nous ont permis d'arriver jusqu'ici. Nous l'adressons également à nos familles pour leur soutien constant et leurs prières, ainsi qu'à nos encadrants et enseignants pour la qualité de leur accompagnement, leur patience et leurs précieux conseils qui nous ont guidés tout au long de ce projet.

Nous exprimons aussi notre gratitude à nos amis et collègues de promotion pour leur entraide, leurs échanges enrichissants et leur bonne humeur qui ont rendu cette aventure plus agréable. Enfin, nous dédions ce travail à toute personne ayant contribué, de près ou de loin, à la réalisation de ce projet.

Avec toute notre reconnaissance et notre sincère gratitude.

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'année.

Nous remercions avant tout notre encadrant académique, M. Tabii Younes, pour sa disponibilité, ses conseils avisés et son soutien constant tout au long de ce travail. Sa rigueur et ses recommandations ont grandement enrichi la qualité de ce projet.

Nous adressons aussi nos sincères remerciements à l'ensemble des enseignants et responsables de la formation pour la qualité de l'enseignement dispensé, qui nous a permis d'acquérir les connaissances nécessaires à la réalisation de ce projet.

Enfin, nous tenons à exprimer notre reconnaissance à nos familles et à nos amis pour leur patience, leur encouragement et leur soutien moral tout au long de cette période exigeante.

Résumé

L'application développée dans le cadre de ce projet de fin d'année, Sihati AI, est une plateforme intelligente de suivi médical qui facilite la communication entre les patients et les professionnels de santé. Elle a pour objectif de centraliser le dossier médical, l'enregistrement des constantes vitales, la prise de rendez-vous et les échanges en temps réel entre patients et médecins.

Conçue comme une solution complète, sécurisée et intuitive, la plateforme comprend une application web développée avec Angular 17, une application mobile Android native, ainsi qu'un backend Spring Boot 3 sécurisé exposant une API REST et des canaux WebSocket pour la communication en temps réel.

La valeur ajoutée du projet réside dans l'intégration de trois modules d'intelligence artificielle : la détection intelligente d'anomalies sur les constantes vitales avec génération d'alertes en temps réel, un avatar médical interactif capable de dialoguer par la voix et l'image avec le patient, ainsi qu'un module de reconnaissance optique (OCR) permettant l'extraction automatique des informations d'une ordonnance scannée.

Le système intègre également une gestion avancée de la sécurité (authentification JWT, chiffrement AES des données sensibles, contrôle d'accès par rôles, conformité RGPD) ainsi que la génération de rapports et de statistiques pour les administrateurs de la plateforme.

Sur le plan technique, l'application a été développée avec Angular 17 côté web, Java 21 / Spring Boot 3.2 côté serveur, une base de données MySQL en production et H2 en développement. Ce projet propose ainsi une solution numérique complète pour le suivi médical, capable d'améliorer la qualité du suivi patient, la réactivité des professionnels de santé et la sécurité des données médicales.

Abstract

This final-year project focuses on the design and development of Sihati AI, an intelligent medical monitoring platform that facilitates communication between patients and healthcare professionals. The main objective is to centralize medical records, vital sign tracking, appointment scheduling and real-time communication between patients and doctors.

The platform includes a web application built with Angular 17, a native Android mobile application, and a secure Spring Boot 3 backend exposing a REST API and WebSocket channels for real-time communication.

The project's added value lies in the integration of three artificial intelligence modules: intelligent anomaly detection on vital signs with real-time alert generation, an interactive medical avatar able to communicate with the patient through voice and image, and an OCR module allowing automatic extraction of information from a scanned prescription.

The system also integrates advanced security features (JWT authentication, AES encryption of sensitive data, role-based access control, GDPR compliance) as well as report and statistics generation for platform administrators.

Developed using Angular 17 for the frontend, Java 21 / Spring Boot 3.2 for the backend, and MySQL/H2 for the database layer, this solution improves the quality of patient monitoring, the responsiveness of healthcare professionals, and the security of medical data.

Table des matières

Avant-propos 3
Dédicaces 4
Remerciements 5
Résumé 6
Abstract 7
Table des matières 8
Liste des abréviations 10
Introduction 11
I. Cahier des charges 12
1. Besoins Fonctionnels 12
1.1. Module Patient 12
1.2. Module Médecin 12
1.3. Module Administration 12
2. Besoins Non-Fonctionnels 12
2.1. Performance 12
2.2. Sécurité 12
2.3. Utilisabilité 12
2.4. Compatibilité 13
3. Règles de Gestion 13
4. Contraintes Techniques 13
5. Livrables Attendus 13
6. Planning Prévisionnel 14
7. Outils et technologies utilisés 15
II. Analyse 16
1. Étude comparative des solutions existantes 16
2. Introduction à l'analyse 16
3. Analyse des besoins fonctionnels 16
a. Suivi des constantes vitales et détection d'anomalies 16
b. Communication patient-médecin 16
c. Gestion des ordonnances (OCR) 16
d. Gestion des rendez-vous 17
4. Analyse des règles de gestion 17
5. Acteurs du système 17
6. Cycle de vie du projet 18
Organisation Scrum 19
III. Conception 20
1. Merise 21
i. Modèle Conceptuel de Données (MCD) 21
ii. Modèle Logique de Données (MLD) 22
2. Diagramme de cas d'utilisation 23
3. Diagramme de classe 24
4. Diagramme d'activité 25

5. Diagramme de séquence 26
6. Diagramme de déploiement 27
IV. Développement 28
1. Espace Patient (Web & Mobile) 29
1.1. Tableau de bord patient 29
1.2. Enregistrement des constantes vitales 30
1.3. Avatar IA vocal (LiveAvatar) 31
1.4. Scan d'ordonnance (OCR) 32
1.5. Prise de rendez-vous et messagerie 33
1.6. Analyse IA des tendances 34
1.7. Gestion des médicaments et des alertes 35
2. Espace Médecin (Web) 36
2.1. Tableau de bord médecin 36
2.2. Gestion des prescriptions 37
2.3. Messagerie et suivi des alertes 38
2.4. Gestion des patients 39
2.5. Prise de rendez-vous (vue médecin) 39
3. Espace Administrateur (Web) 40
3.1. Gestion des utilisateurs 40
3.2. Audit de sécurité 40
3.3. Statistiques et rapports 41
3.4. Création de comptes 41
4. Espace Urgentiste (accès d'urgence) 42
5. Backend et API REST 43
6. Sécurité 43
7. Application Mobile (Android) 44
V. Tests & Validation 52
1. Stratégie de test 52
2. Cas de test représentatifs 52
3. Synthèse des résultats 52
VI. Gestion des risques 53
VII. Bilan des objectifs atteints 54
1. Besoins fonctionnels 54
2. Besoins non fonctionnels 54
Conclusion 56
Références 57

Liste des abréviations

- **API :** Application Programming Interface
- **BF :** Besoin Fonctionnel
- **BNF :** Besoin Non-Fonctionnel
- **CRUD :** Create, Read, Update, Delete
- **CSS :** Cascading Style Sheets
- **IA :** Intelligence Artificielle
- **JS :** JavaScript
- **JWT :** JSON Web Token
- **KPI :** Key Performance Indicator
- **MCD :** Modèle Conceptuel de Données
- **MLD :** Modèle Logique de Données
- **OCR :** Optical Character Recognition
- **ORM :** Object-Relational Mapping
- **OWASP :** Open Web Application Security Project
- **PDF :** Portable Document Format
- **RBAC :** Role-Based Access Control
- **RGPD :** Règlement Général sur la Protection des Données
- **SGBD :** Système de Gestion de Base de Données
- **SQL :** Structured Query Language
- **UML :** Unified Modeling Language

Introduction

Dans un contexte où la qualité du suivi médical et la rapidité d'accès à l'information de santé sont des critères essentiels pour améliorer la prise en charge des patients, les établissements et professionnels de santé cherchent à moderniser leurs outils de gestion. Aujourd'hui, la digitalisation s'impose comme une solution incontournable pour optimiser le suivi médical, réduire les erreurs humaines et améliorer la réactivité face aux situations à risque.

Notre projet s'inscrit dans cette démarche de modernisation en proposant le développement d'une plateforme intelligente de suivi médical, Sihati AI. Cette plateforme permettra notamment aux patients de gérer leur dossier médical, d'enregistrer leurs constantes vitales, de prendre rendez-vous et de communiquer en temps réel avec leur médecin, tout en bénéficiant d'un accompagnement intelligent grâce à l'intégration de modules d'intelligence artificielle.

L'objectif de ce travail est de concevoir et de réaliser une solution intuitive, sécurisée et performante qui facilite la gestion quotidienne du suivi de santé, tout en offrant aux patients et aux professionnels de santé un moyen simple, rapide et fiable de collaborer.

Le projet a été développé en équipe et couvre l'ensemble des étapes classiques d'un cycle de développement logiciel : analyse des besoins, conception, développement, tests et documentation.

Ce rapport décrit en détail toutes les étapes du projet, des objectifs initiaux jusqu'à la réalisation finale, en passant par la modélisation, le choix des technologies et la présentation de l'architecture développée.

I. Cahier des charges

Sihati AI est une plateforme intelligente de suivi médical destinée à faciliter la communication entre patients et professionnels de santé, en combinant sécurité des données, intelligence artificielle et expérience utilisateur multi-plateforme (Web, Mobile Android, API REST).

1. Besoins Fonctionnels

1.1. Module Patient

- **BF1 :** Gestion du dossier médical personnel (antécédents, allergies, maladies chroniques)
- **BF2 :** Enregistrement des constantes vitales : tension artérielle, fréquence cardiaque, SpO2, glycémie, température, poids
- **BF3 :** Réception d'alertes automatiques en cas de valeur critique (NORMAL / WARNING / CRITICAL)
- **BF4 :** Interaction avec l'Avatar IA vocal (LiveAvatar) pour des conseils de santé en temps réel
- **BF5 :** Scan d'ordonnances par caméra, image ou PDF avec extraction automatique (OCR)
- **BF6 :** Prise de rendez-vous en ligne avec un médecin
- **BF7 :** Messagerie sécurisée en temps réel avec le médecin (WebSocket)
- **BF8 :** Bouton SOS d'urgence

1.2. Module Médecin

- **BF9 :** Tableau de bord listant l'ensemble des patients suivis
- **BF10 :** Visualisation en temps réel des alertes critiques
- **BF11 :** Consultation de l'historique et des graphiques d'évolution de chaque patient
- **BF12 :** Messagerie sécurisée avec les patients
- **BF13 :** Gestion et émission de prescriptions médicales

1.3. Module Administration

- **BF14 :** Gestion globale des utilisateurs et des rôles de la plateforme
- **BF15 :** Audit de sécurité complet (journal des connexions et des actions)
- **BF16 :** Génération de statistiques et de rapports d'utilisation

2. Besoins Non-Fonctionnels

2.1. Performance

- **BNF1 :** Temps de réponse API inférieur à 200 ms pour 95% des requêtes
- **BNF2 :** Support de plusieurs centaines d'utilisateurs simultanés
- **BNF3 :** Disponibilité du service supérieure à 99,5% du temps

2.2. Sécurité

- **BNF4 :** Authentification JWT avec option de double authentification (2FA par OTP email)
- **BNF5 :** Chiffrement AES-128 des données médicales sensibles (allergies, maladies chroniques)
- **BNF6 :** Conformité RGPD et bonnes pratiques HIPAA (droit à l'oubli, audit trail)
- **BNF7 :** Sauvegarde automatique quotidienne de la base de données

2.3. Utilisabilité

- **BNF8 :** Interface responsive (mobile, tablette, ordinateur)
- **BNF9 :** Navigation intuitive avec un minimum de clics pour les actions principales

- **BNF10 :** Support multilingue (français, arabe, anglais)

2.4. Compatibilité

- **BNF11 :** Support des dernières versions de Chrome, Firefox, Safari, Edge
- **BNF12 :** Export des données en PDF et Excel
- **BNF13 :** Compatibilité Android à partir de l'API 24 (minSdk 24, targetSdk 34)

3. Règles de Gestion

Règle de gestion	Besoin fonctionnel concerné	Complexité
RG1 : Une tension systolique > 180 mmHg déclenche automatiquement le statut CRITICAL	BF2, BF3	Difficile
RG2 : Une glycémie < 70 mg/dL déclenche automatiquement le statut CRITICAL	BF2, BF3	Difficile
RG3 : Une saturation en oxygène (SpO2) < 90% déclenche automatiquement le statut CRITICAL	BF2, BF3	Moyenne
RG4 : Un compte est bloqué après 5 tentatives de connexion échouées en 15 minutes	BF sécurité	Moyenne
RG5 : Seul un médecin peut émettre ou modifier une prescription	BF13	Facile
RG6 : Les données sensibles (allergies, maladies chroniques) sont chiffrées en base (AES)	BF1	Moyenne
RG7 : Toute action sensible est historisée dans la table AuditLog	BF15	Moyenne
RG8 : Un rendez-vous ne peut être pris que sur un créneau disponible du médecin	BF6	Moyenne

4. Contraintes Techniques

- **Backend :** Spring Boot 3.2.0, Java 21, Maven, API REST + WebSocket (STOMP/SockJS)
- **Frontend Web :** Angular 17 (standalone components), Angular Material, RxJS, Chart.js
- **Mobile :** Android natif (Java), Retrofit 2.9, Material Design 3, minSdk 24 / targetSdk 34
- **Base de données :** MySQL 8.0 en production, H2 en développement/tests, Hibernate ORM / JPA
- **IA / OCR :** Tesseract OCR, Apache PDFBox 3.0.1 pour l'extraction PDF
- **Sécurité :** JWT (JJWT 0.12.3, HMAC-SHA256), Spring Security, chiffrement AES-128
- **DevOps :** Docker Compose (MySQL + Spring Boot + Angular/nginx), GitHub, Swagger UI

5. Livrables Attendus

- Code source complet (GitHub) : backend Spring Boot, frontend Angular, application Android
- API REST documentée (Swagger UI)
- Base de données avec scripts d'initialisation et jeux de données de test
- Suite de tests unitaires (JUnit 5 + Mockito)
- Diagrammes UML (StarUML) et modèles Merise
- Rapport de projet (~60 pages) et présentation PowerPoint (12-15 slides)
- Ordonnance de test pour la démonstration du module OCR

6. Planning Prévisionnel

Phase	Durée	Livrables
P1 : Setup & Authentification	2-3 semaines	Projet Spring Boot, JWT, base de données, Swagger
P2 : Fonctionnalités cœur	4-5 semaines	CRUD constantes, alertes, rendez-vous, messagerie
P3 : Frontend Web	4-5 semaines	Interfaces Angular, tableaux de bord, graphiques
P4 : Mobile	4-5 semaines	Application Android, connexion à l'API
P5 : IA & OCR	3-4 semaines	Détection d'anomalies, avatar IA, OCR ordonnances
P6 : Tests & Déploiement	2-3 semaines	Tests, sécurité, mise en production

Le planning ci-dessus est représenté sous forme de diagramme de Gantt, qui met en évidence l'enchaînement des six phases ainsi que les sprints agiles qui les traversent.

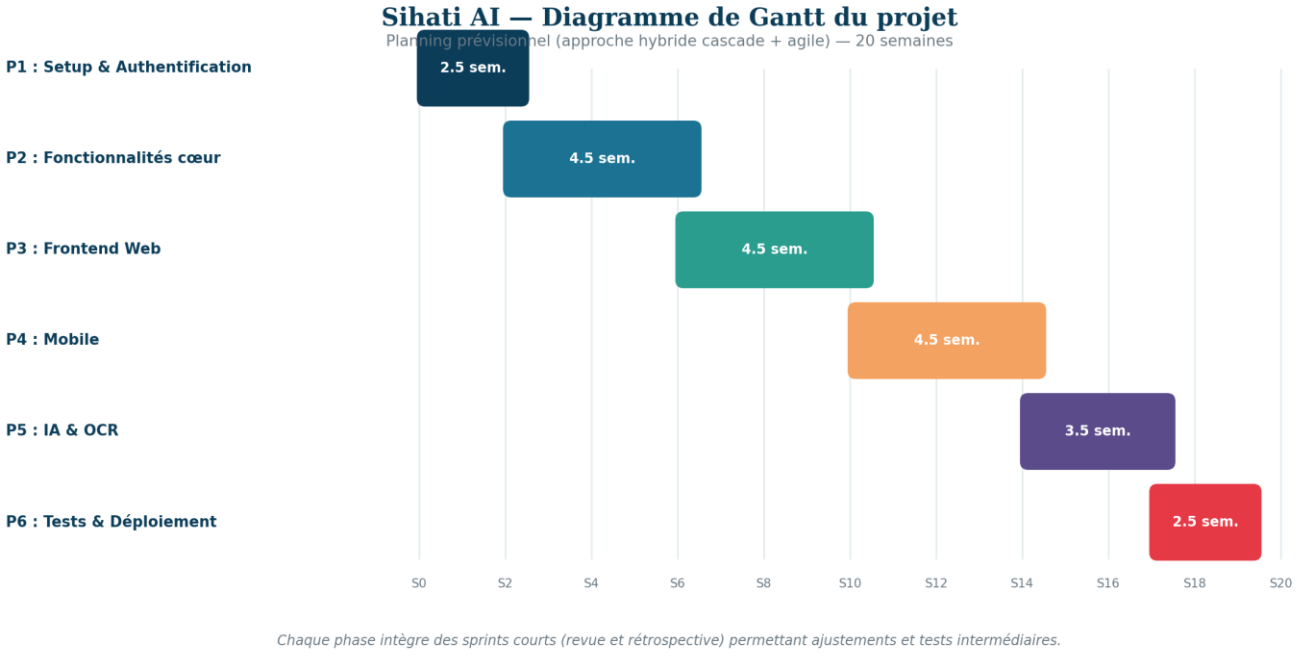


Figure 37 : Diagramme de Gantt du planning prévisionnel

7. Outils et technologies utilisés

La plateforme Sihati AI repose sur un socle technique moderne, robuste et largement adopté dans l'industrie :

Java 21 Langage backend, robustesse et performance	**Spring Boot 3.2** Framework REST + sécurité + logique métier	**Angular 17** Framework frontend réactif et modulaire
TypeScript Typage statique pour un code fiable	**MySQL** Base de données relationnelle sécurisée	**Tesseract OCR** Extraction de texte depuis les ordonnances
JWT Authentification et sécurisation des accès	**GitHub** Gestion de version et collaboration du code	**Swagger** Documentation et test des API REST
Postman Test et validation des requêtes API	**Chart.Js** Visualisation de données et graphiques interactifs	**Docker** Conteneurisation et déploiement de l'application

La répartition ci-dessous synthétise le poids de chaque couche technique dans la plateforme : le backend et le frontend/mobile forment le socle applicatif, complétés par les outils DevOps, la base de données et le module d'intelligence artificielle.

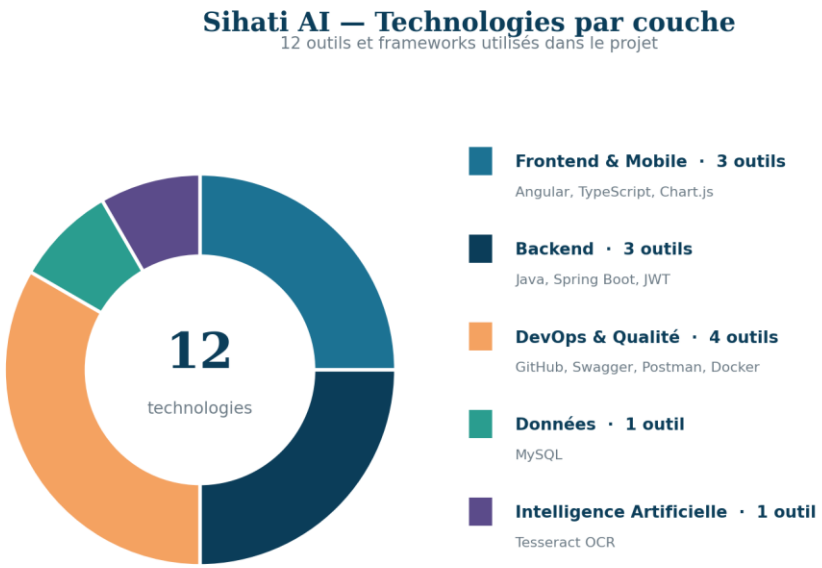


Figure 38 : Répartition des technologies par couche

II. Analyse

1. Étude comparative des solutions existantes

Avant de développer notre propre solution, nous avons analysé plusieurs systèmes existants de suivi médical numérique. La plupart se limitent à la gestion administrative des rendez-vous et des dossiers, sans offrir de véritable intelligence artificielle ni d'expérience utilisateur intégrée. Sihati AI se distingue par une couverture fonctionnelle beaucoup plus large : un module complet pour chaque acteur (patient, médecin, administrateur), une détection intelligente des anomalies médicales en temps réel, un avatar médical conversationnel, ainsi qu'un module OCR pour l'extraction automatique des ordonnances. La plateforme intègre également des mécanismes de sécurité avancés (JWT, chiffrement AES, audit trail) rarement présents dans les solutions académiques comparables.

Critère	Dossier papier / Excel	Apps santé basiques	Sihati AI
Centralisation des données	Non	Partielle	Oui, dossier médical unique
Alertes automatiques	Non	Rares	Oui, IA en temps réel
Communication patient-médecin	Téléphone	Chat texte simple	Chat WebSocket + Avatar vocal
Extraction d'ordonnances	Manuelle	Non disponible	OCR automatique (PDF/Image/Caméra)
Sécurité des données	Faible	Variable	JWT, AES, RBAC, audit, OWASP

2. Introduction à l'analyse

L'analyse est une étape fondamentale du projet, car elle permet de traduire les besoins exprimés dans le cahier des charges en spécifications claires et détaillées. Elle vise à comprendre les fonctionnalités attendues, à identifier les acteurs du système et à définir les règles de gestion qui guideront la conception et le développement de la plateforme.

3. Analyse des besoins fonctionnels

a. Suivi des constantes vitales et détection d'anomalies

- **Problème métier :** le suivi manuel des constantes de santé retarde la détection des situations à risque.
- **Solution informatique :** un service d'analyse (AnomalyDetectionService) compare chaque mesure saisie à des seuils médicaux prédéfinis.
- **Traitement :** chaque mesure reçoit un statut (NORMAL, WARNING, CRITICAL). Une alerte est automatiquement créée et diffusée par WebSocket au patient et au médecin en moins d'une seconde.

b. Communication patient-médecin

- **Problème métier :** les échanges par téléphone sont peu tracés et peu réactifs en cas d'urgence.
- **Solution informatique :** une messagerie sécurisée en temps réel (WebSocket/STOMP) reliée au profil de chaque patient et médecin.
- **Traitement :** chaque message est horodaté et stocké ; un statut de lecture (readByReceiver) permet de suivre la prise en charge.

c. Gestion des ordonnances (OCR)

- **Problème métier :** la ressaisie manuelle des médicaments prescrits est source d'erreurs et chronophage.
- **Solution informatique :** un module OCR (Tesseract pour les images, PDFBox pour les PDF) extrait automatiquement médicaments, dosages et posologies.
- **Traitement :** les informations extraites sont proposées à l'utilisateur pour import en un clic dans son dossier médicament.

d. Gestion des rendez-vous

- **Problème métier :** la prise de rendez-vous par téléphone entraîne des doublons et des erreurs de planification.
- **Solution informatique :** un formulaire en ligne présente les créneaux disponibles du médecin choisi.
- **Traitement :** chaque rendez-vous est enregistré avec un statut (en attente, confirmé, annulé) et notifié aux deux parties.

4. Analyse des règles de gestion

- Les seuils critiques (tension, glycémie, SpO2) sont centralisés dans le service d'analyse et appliqués uniformément à chaque nouvelle mesure.
- Le chiffrement AES des champs sensibles (allergies, maladies chroniques) est appliqué avant toute persistance en base de données.
- Le contrôle d'accès (RBAC) restreint chaque endpoint de l'API à un ou plusieurs rôles autorisés (PATIENT, DOCTOR, ADMIN, EMERGENCY).
- Toute tentative de connexion échouée est comptabilisée ; au-delà de 5 essais en 15 minutes, le compte est temporairement bloqué.

5. Acteurs du système

Patient : il gère son dossier médical, enregistre ses constantes vitales, prend rendez-vous, échange avec l'Avatar IA et avec son médecin, et reçoit des alertes en cas d'anomalie.

Médecin : il consulte le tableau de bord de ses patients, analyse leur état de santé, gère les prescriptions et répond aux alertes critiques.

Administrateur : il supervise l'ensemble de la plateforme, gère les comptes utilisateurs, consulte l'audit de sécurité et génère les statistiques globales.

Le diagramme de cas d'utilisation (section III.2) dénombre 24 cas d'utilisation répartis entre les trois acteurs principaux et l'authentification partagée. Le graphique suivant illustre cette répartition.

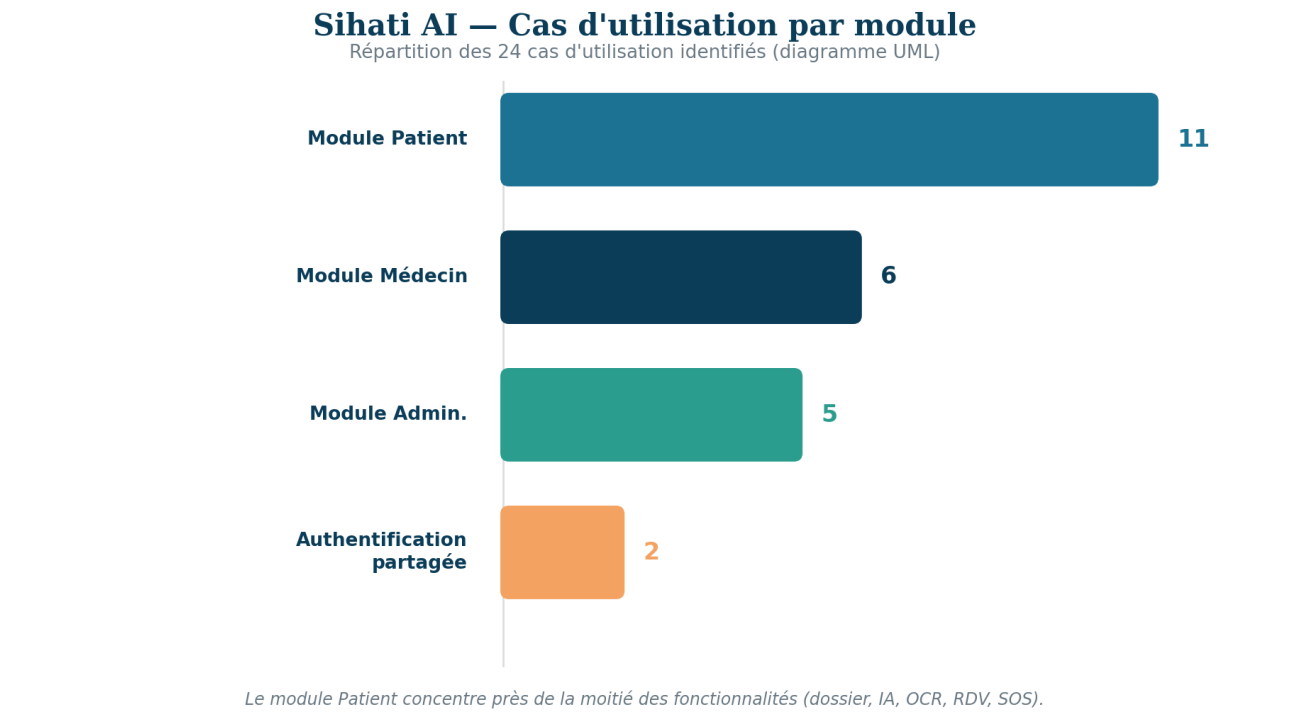


Figure 39 : Répartition des cas d'utilisation par module

6. Cycle de vie du projet

Pour la réalisation de ce projet, nous avons adopté une approche hybride combinant le cycle en cascade pour la structuration académique (analyse, conception, développement, tests, déploiement) et des principes issus des méthodes agiles (itérations courtes, tests intermédiaires, ajustements progressifs), afin de garantir à la fois une bonne visibilité sur l'avancement et une capacité de correction rapide des anomalies.

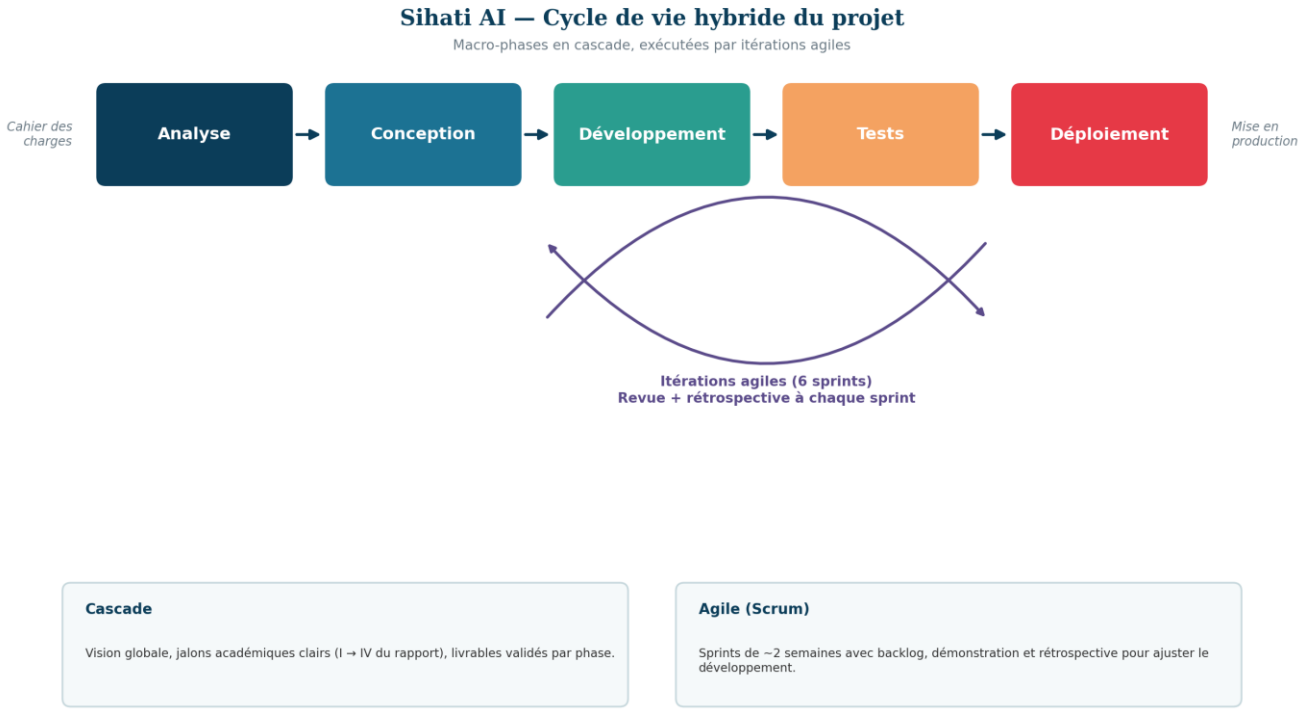


Figure 40 : Cycle de vie hybride du projet

Organisation Scrum

Le développement a été organisé en six sprints, chacun correspondant à une phase du planning (P1 à P6). Chaque sprint suit le rythme classique des cérémonies Scrum : planification en début de sprint, points d'avancement courts entre les membres de l'équipe, revue de sprint avec démonstration à l'encadrant, puis rétrospective pour ajuster la méthode de travail avant le sprint suivant.

L'équipe, réduite à trois personnes, adapte les rôles Scrum classiques : l'encadrant M. Tabii Younes joue le rôle de Product Owner en priorisant le backlog et en validant chaque sprint, tandis que le rôle de Scrum Master tourne entre les trois étudiantes selon les sprints. Les trois membres forment ensemble l'équipe de développement, responsable de la conception, du développement et des tests.



Figure 41 : Organisation des sprints Scrum de l'équipe

III. Conception

L'architecture technique adoptée pour ce projet repose sur un modèle client-serveur structuré et sécurisé.

- **Frontend Web :** conçu avec Angular 17 (standalone components) et Angular Material, il offre une interface moderne, responsive et intuitive, adaptée aux trois profils d'utilisateurs.
- **Mobile :** application Android native permettant au patient d'accéder à son suivi médical depuis son smartphone.
- **Backend :** développé avec Spring Boot 3, il centralise la logique métier, applique les règles de gestion et de sécurité, et expose une API REST ainsi que des canaux WebSocket (STOMP/SockJS) pour la communication en temps réel.
- **Base de données :** MySQL en production et H2 en développement, utilisée pour stocker les utilisateurs, dossiers médicaux, alertes, rendez-vous et messages.
- **Modules IA :** détection d'anomalies, avatar médical vocal (LiveAvatar) et OCR médical, intégrés au backend et appelés depuis le frontend.

L'architecture ainsi conçue garantit la modularité, l'évolutivité et la maintenabilité de la plateforme, tout en assurant la cohérence des traitements, la sécurité des données médicales et la performance globale du système.

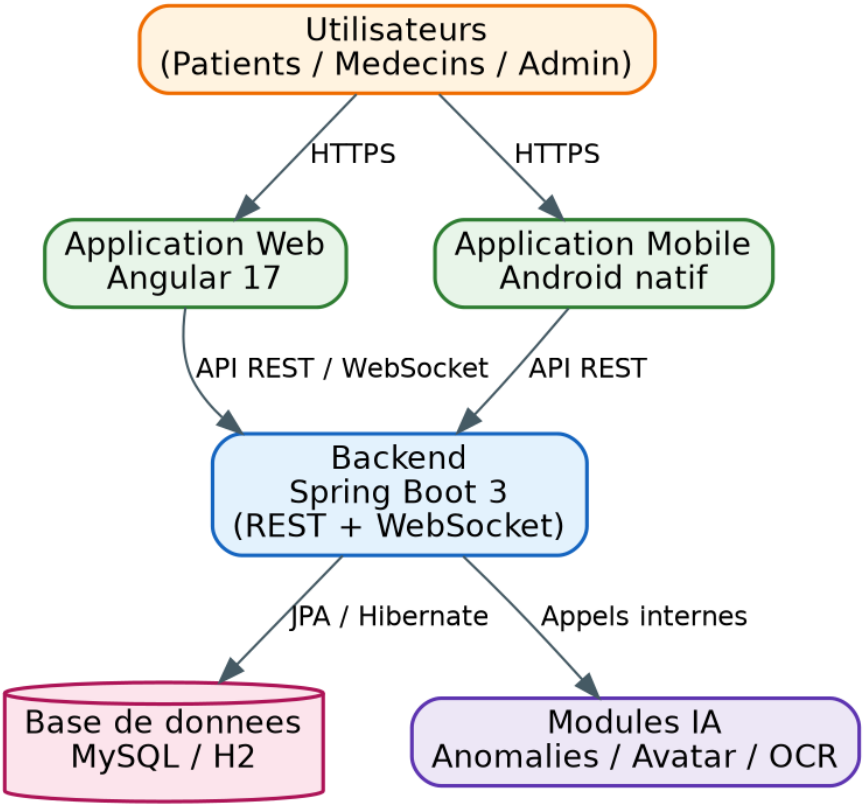


Figure 1 : Schéma de l'architecture technique

1. Merise

i. Modèle Conceptuel de Données (MCD)

Le MCD représente les entités principales du système ainsi que les relations entre elles. Il permet de comprendre la structure logique des données manipulées : utilisateurs, patients, médecins, constantes de santé, alertes, rendez-vous, médicaments, messagerie et audit.

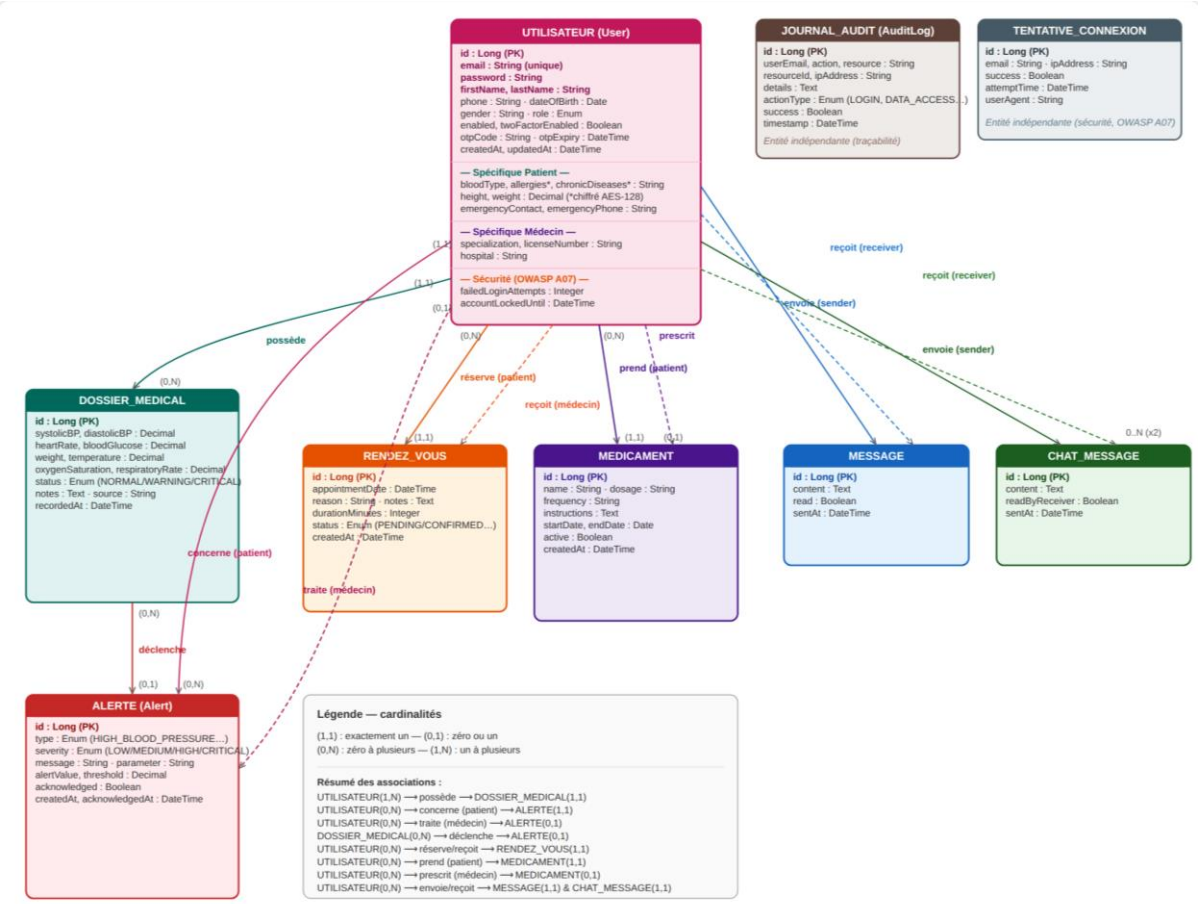


Figure 2 : Modèle Conceptuel de Données (MCD)

ii. Modèle Logique de Données (MLD)

Le MLD traduit le MCD en tables relationnelles exploitables par le SGBD. Chaque entité devient une table, et les relations sont transformées en clés étrangères :

Table	Champs (PK soulignée, #FK en italique bleu)
Utilisateur	**id**, email, mot_de_passe, prenom, nom, telephone, date_naissance, sexe, role, actif, authentification_2fa, code_otp, expiration_otp, date_creation, date_modification, groupe_sanguin, taille, poids, allergies, maladies_chroniques, contact_urgence, telephone_urgence, specialite, numero_licence, hopital, tentatives_connexion, compte_bloque_jusqu_a
DossierMedical	**id**, *#patient_id*, pression_systolique, pression_diastolique, frequence_cardiaque, glycemie, poids, temperature, saturation_oxygene, frequence_respiratoire, statut, notes, source, date_enregistrement
Alerte	**id**, *#patient_id*, *#medecin_id*, *#dossier_medical_id*, type, severite, message, parametre, valeur_alerte, seuil, acquittee, date_creation, date_acquittement
RendezVous	**id**, *#patient_id*, *#medecin_id*, date_rendez_vous, motif, notes, duree_minutes, statut, date_creation
Medicament	**id**, *#patient_id*, *#medecin_id*, nom, dosage, frequence, instructions, date_debut, date_fin, actif, date_creation
Message	**id**, *#expediteur_id*, *#destinataire_id*, contenu, lu, date_envoi
ChatMessage	**id**, *#expediteur_id*, *#destinataire_id*, contenu, lu_par_destinataire, date_envoi
JournalAudit	**id**, email_utilisateur, action, ressource, ressource_id, adresse_ip, details, type_action, succes, date_heure
TentativeConnexion	**id**, email, adresse_ip, succes, date_tentative, user_agent

Légende : le champ souligné en gras est la clé primaire (PK) de la table ; les champs précédés de # et en italique bleu sont des clés étrangères (FK) référençant la clé primaire d'une autre table.

2. Diagramme de cas d'utilisation

Ce diagramme illustre les différents acteurs du système (patient, médecin, administrateur) et les cas d'utilisation auxquels ils participent. Il permet d'identifier les fonctionnalités principales attendues de la plateforme.

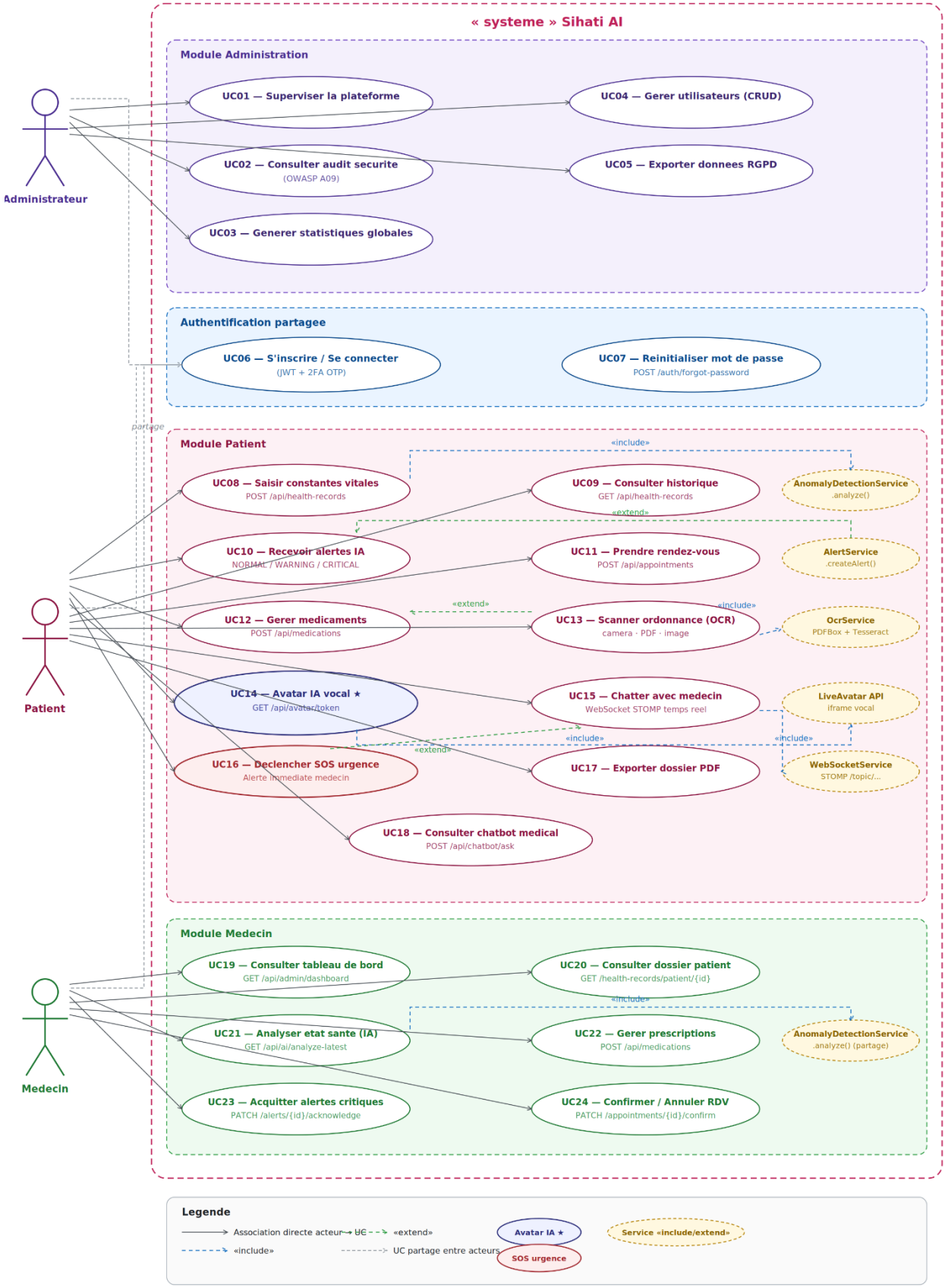


Figure 3 : Diagramme de cas d'utilisation

3. Diagramme de classe

Le diagramme de classes décrit la structure statique du système à travers les classes, leurs attributs et méthodes, ainsi que les relations entre elles. Il constitue la base du développement orienté objet du backend Spring Boot.



Figure 4 : Diagramme de classe

4. Diagramme d'activité

Ce diagramme représente le processus métier de détection d'anomalie sur une constante vitale, depuis la saisie par le patient jusqu'à la réaction du médecin, en mettant en évidence les conditions et décisions prises par le système.

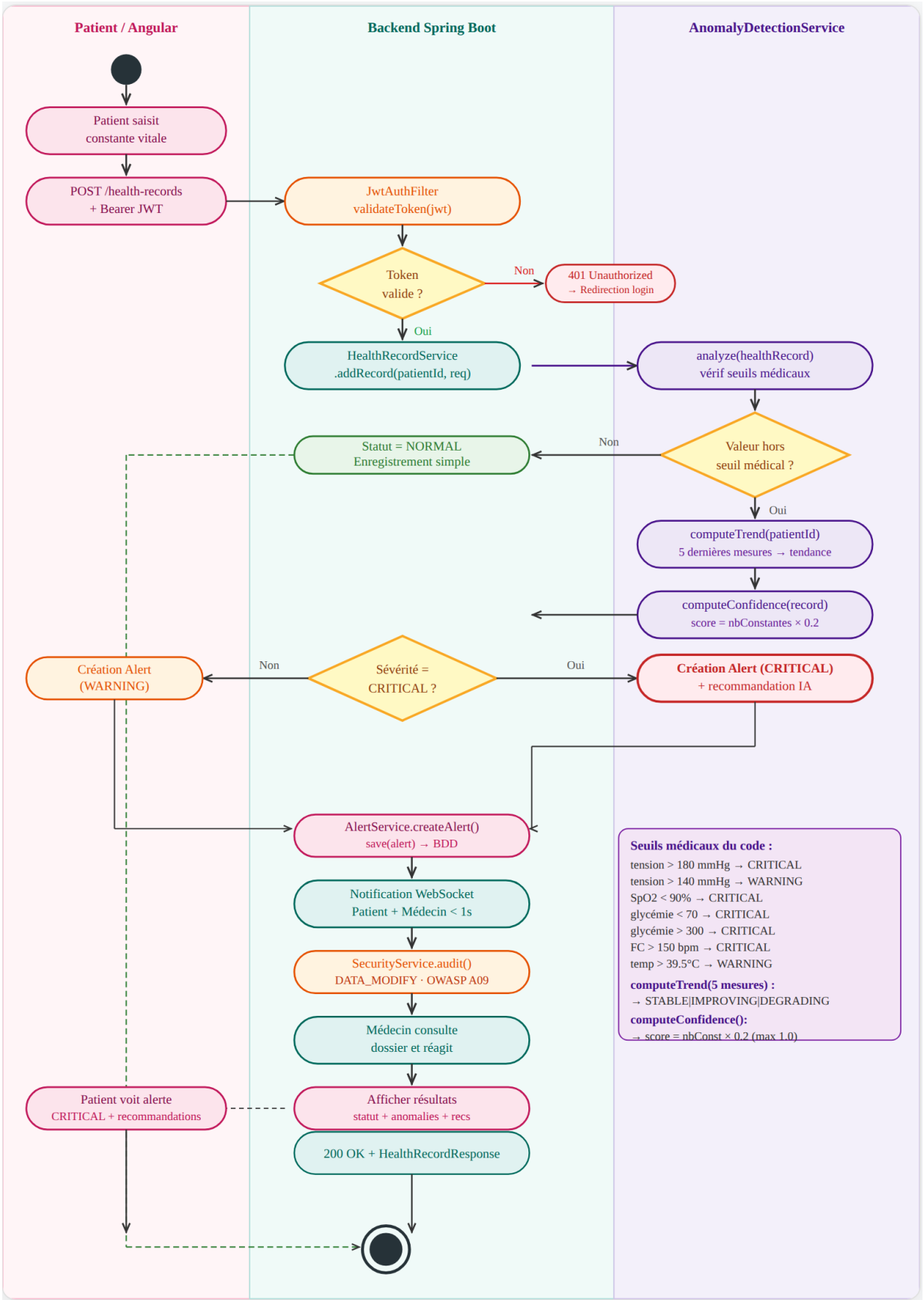


Figure 5 : Diagramme d'activité — Détection d'anomalie

5. Diagramme de séquence

Le diagramme de séquence illustre les interactions dynamiques entre le patient, l'application, le backend, la base de données et le médecin lors de la saisie d'une constante critique. Il met en avant l'ordre chronologique des messages échangés, notamment la notification en temps réel via WebSocket.

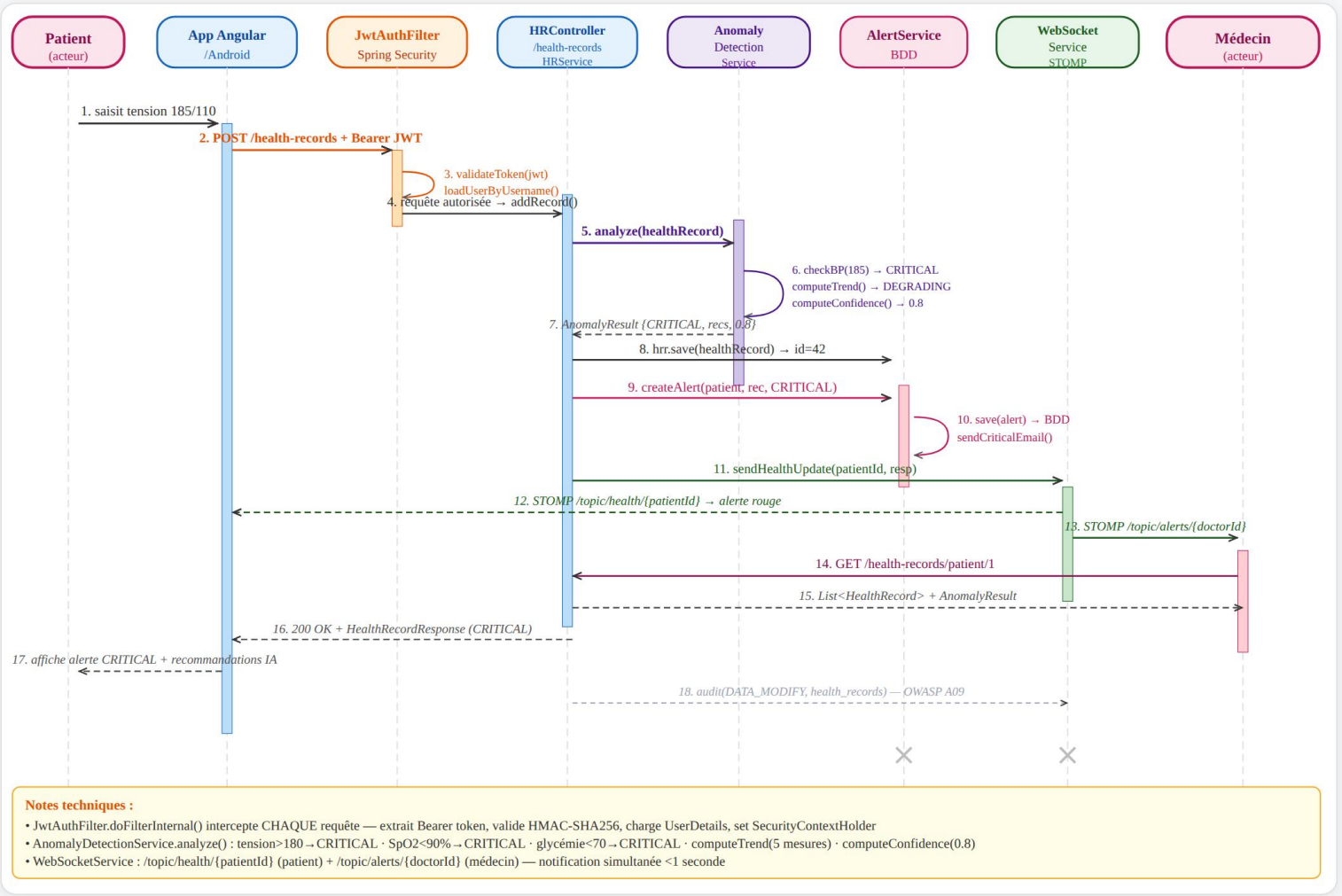


Figure 6 : Diagramme de séquence — Alerte critique en temps réel

6. Diagramme de déploiement

Le diagramme de déploiement ci-dessous représente l'architecture physique de la plateforme : les postes clients (navigateur web et smartphone Android) communiquent en HTTPS/WSS avec le serveur d'application, qui héberge l'API Spring Boot (REST + WebSocket), le module de sécurité et les modules d'intelligence artificielle. Le serveur dialogue avec la base de données MySQL via JDBC et avec le service cloud externe LiveAvatar pour l'avatar IA vocal.

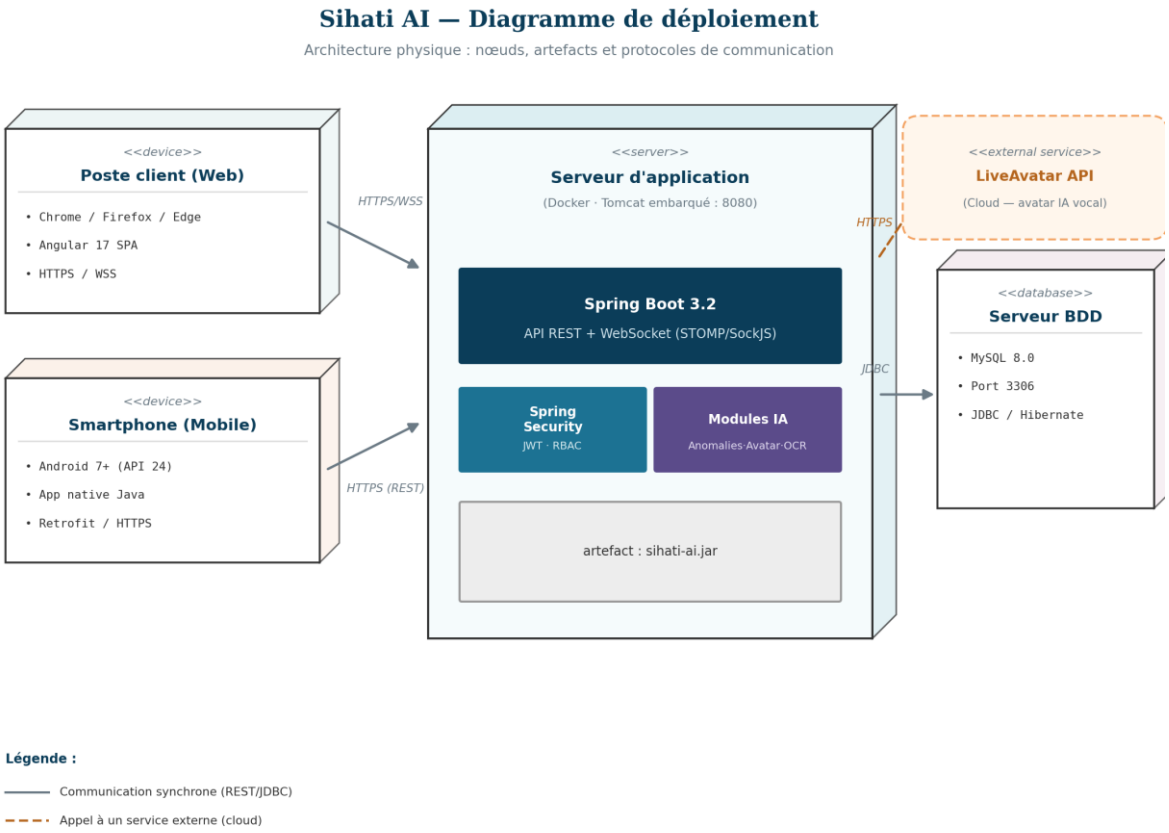


Figure 42 : Diagramme de déploiement de Sihati AI

IV. Développement

Dans cette partie, nous passons de la conception théorique à la réalisation concrète de la plateforme Sihati AI. Le développement a été mené en suivant une approche modulaire, où chaque fonctionnalité a été isolée et construite progressivement afin de faciliter la maintenance et l'évolution future du système.

Le back-end a été implémenté avec Spring Boot 3.2.0 (Java 21), exposant une API REST documentée via Swagger ainsi que des canaux WebSocket pour la communication en temps réel. La base de données MySQL centralise toutes les informations relatives aux patients, médecins, constantes de santé et messages. Le front-end web a été réalisé avec Angular 17 et Angular Material afin d'offrir une interface moderne, responsive et intuitive, tandis que l'application Android native permet un accès mobile complet aux fonctionnalités patient.

Nous présentons ci-après les principaux modules développés, en commençant par l'espace Patient.

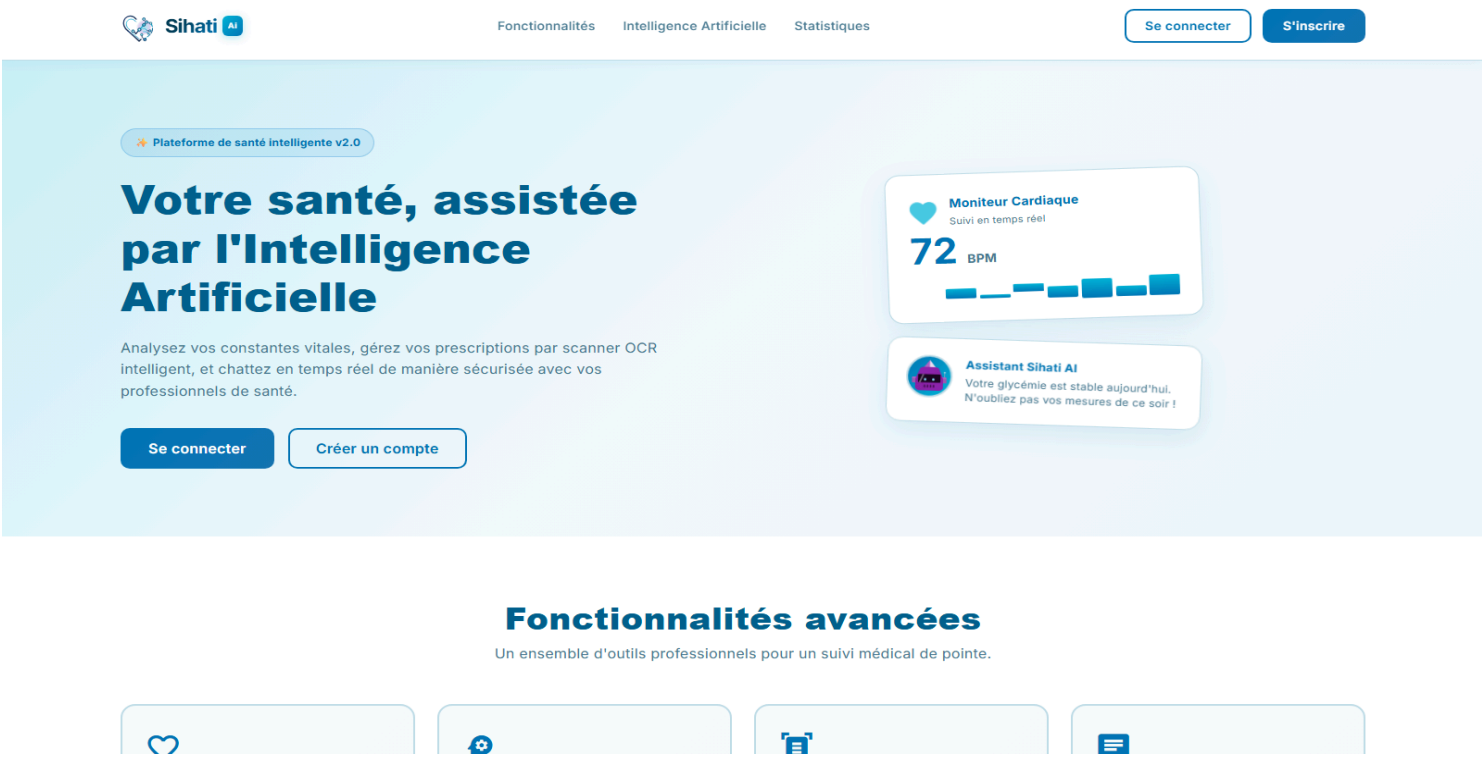


Figure 7 : Page d'accueil de la plateforme Sihati AI

1. Espace Patient (Web & Mobile)

1.1. Tableau de bord patient

Le tableau de bord patient centralise, dès la connexion, les informations essentielles : dernière constante enregistrée, statut de santé courant (NORMAL/WARNING/CRITICAL), prochain rendez-vous et messages non lus. Un bouton SOS bien visible permet de déclencher une alerte d'urgence en un clic.

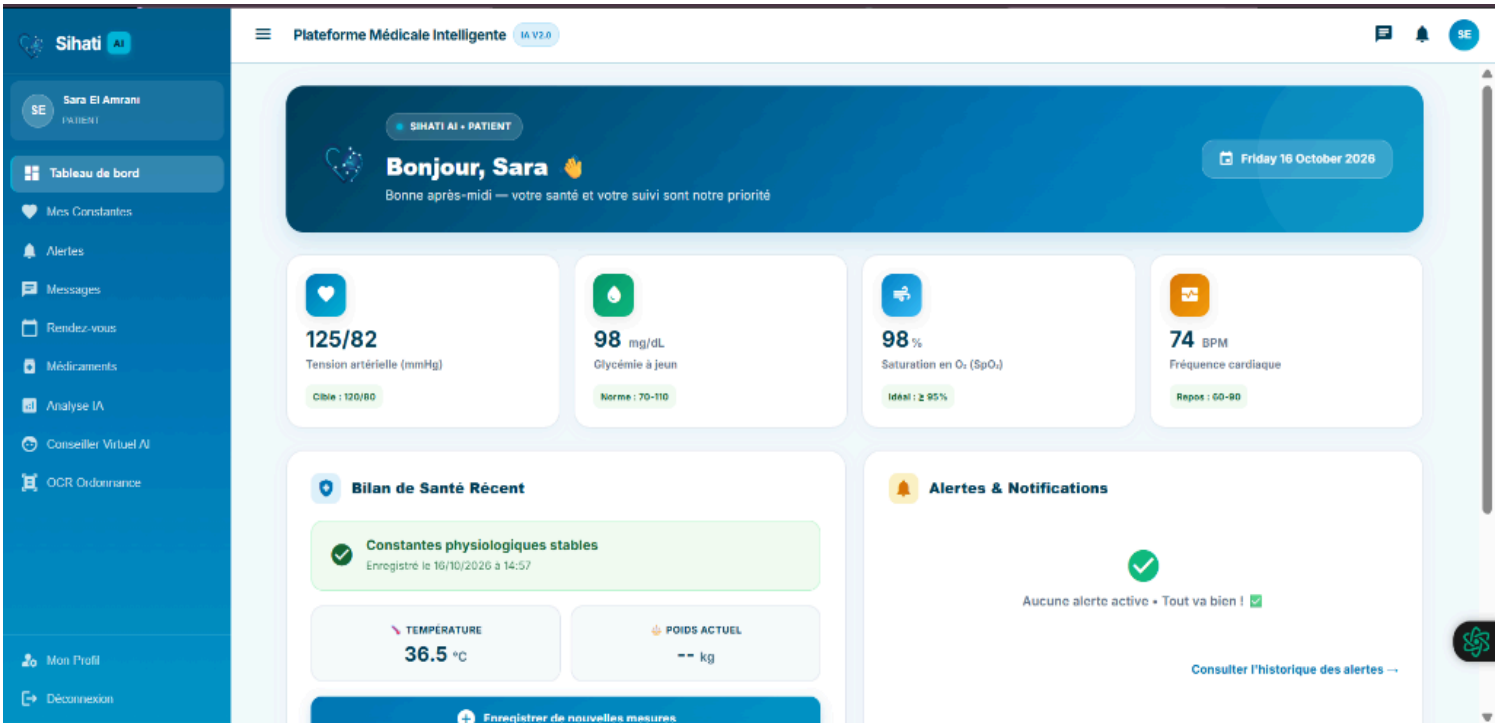


Figure 8 : Tableau de bord patient

1.2. Enregistrement des constantes vitales

Cette interface permet au patient de saisir tension artérielle, fréquence cardiaque, SpO2, glycémie, température et poids. Chaque saisie est immédiatement transmise au service d'analyse d'anomalies (AnomalyDetectionService), qui calcule le statut et affiche instantanément un retour visuel (vert/orange/rouge) ainsi qu'une recommandation générée automatiquement.

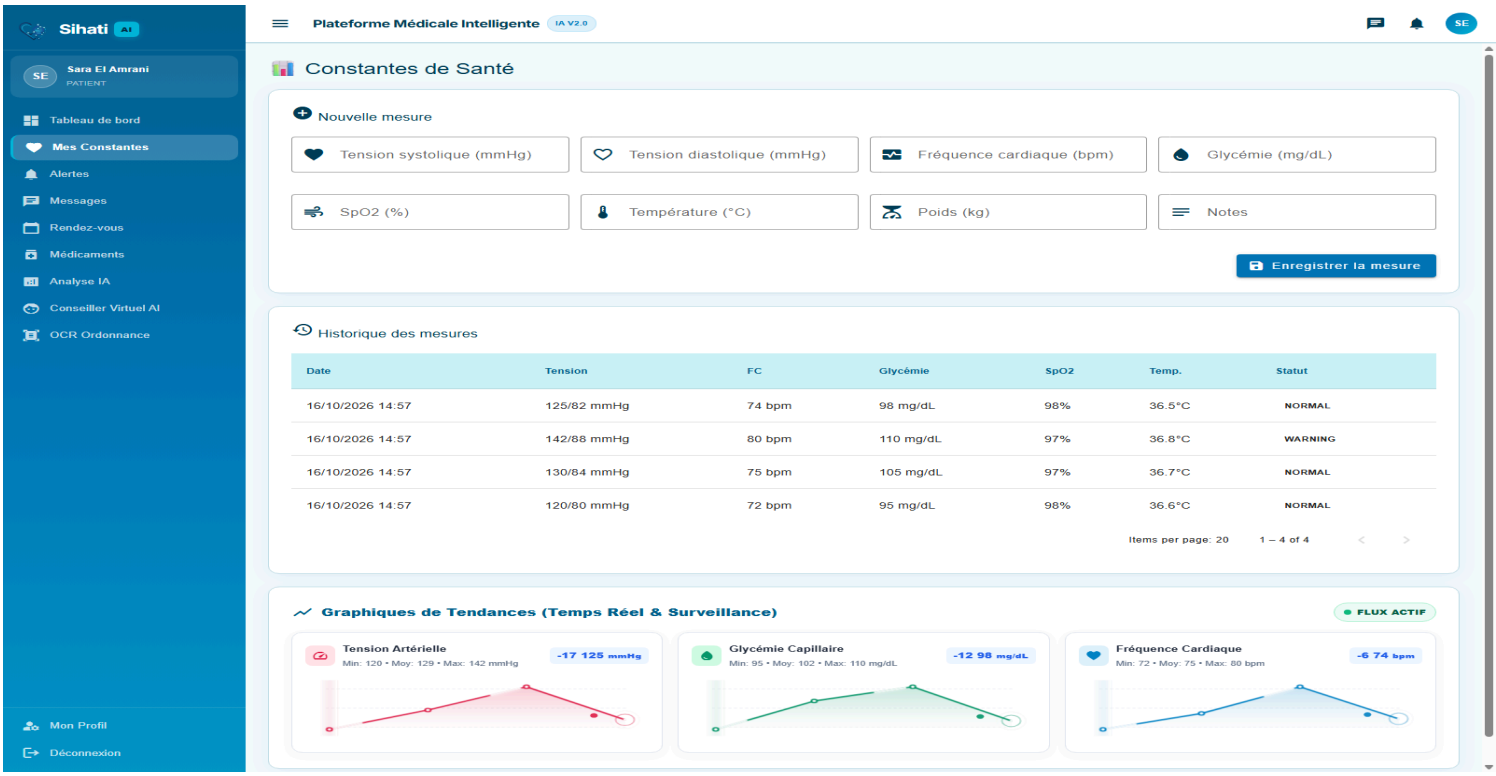


Figure 9 : Enregistrement des constantes vitales

1.3. Avatar IA vocal (LiveAvatar)

Le module Avatar IA propose une expérience conversationnelle par la voix : le patient pose une question à l'oral, le contexte (profil et dernières mesures) est injecté dans la requête, et l'avatar répond oralement avec un avertissement médical systématique rappelant qu'il ne remplace pas un avis médical professionnel. Ce module est disponible 24h/24 sans rendez-vous.

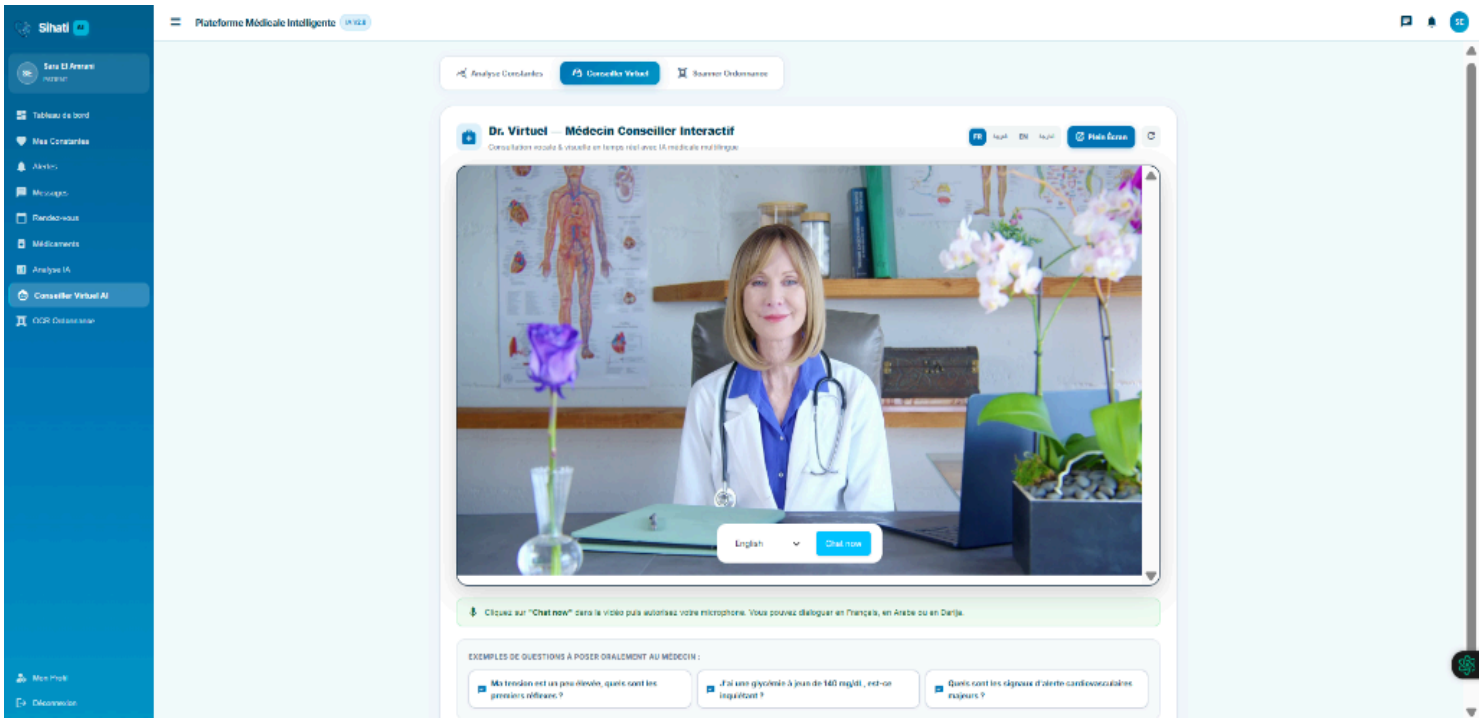


Figure 10 : Conseiller santé virtuel (Avatar IA)

1.4. Scan d'ordonnance (OCR)

Le patient peut soumettre une ordonnance par upload de PDF, upload d'image ou capture directe via la caméra (navigator.mediaDevices sur le web, Camera2 API sur Android). Le backend traite le document avec PDFBox ou Tesseract selon le format, puis propose une extraction structurée (médicaments, dosages, posologies, médecin prescripteur, date) importable en un clic dans la liste des médicaments du patient.

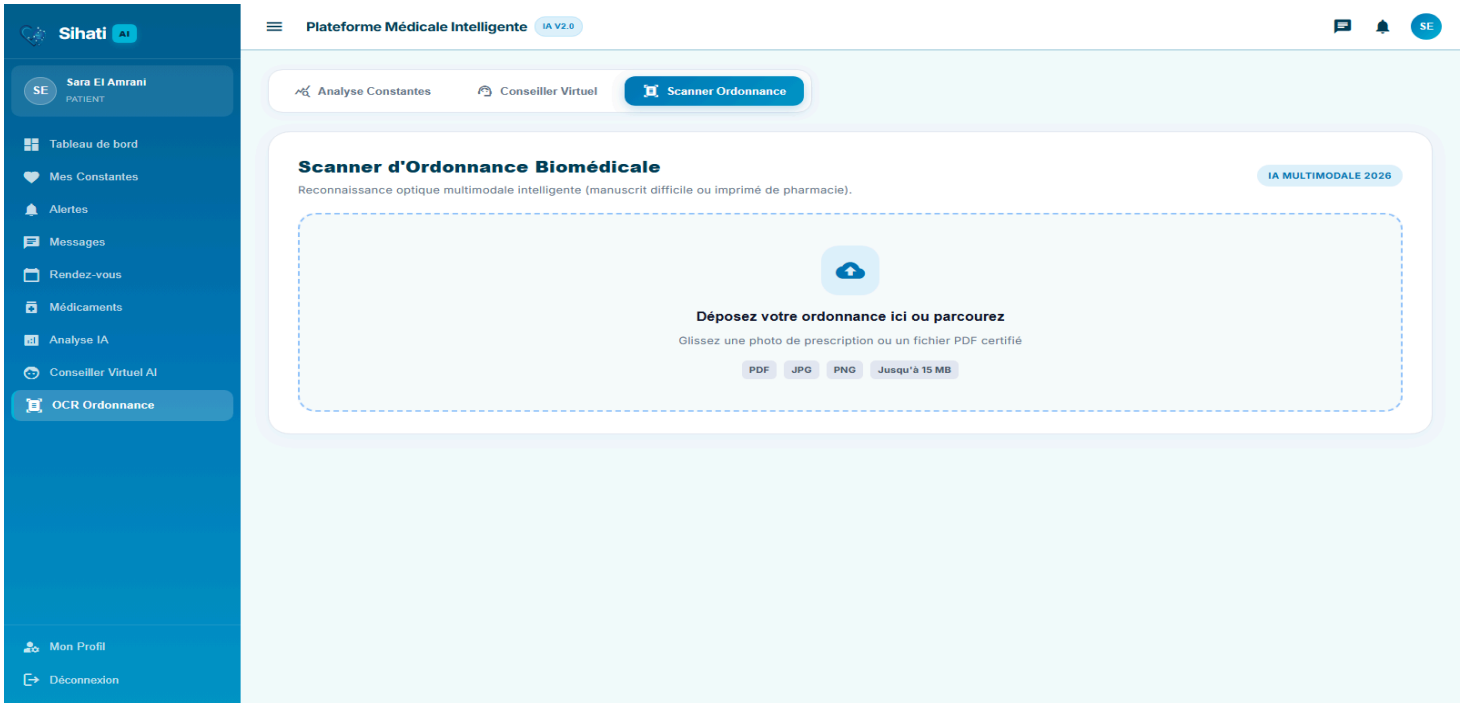


Figure 11 : Analyse d'ordonnance par OCR

1.5. Prise de rendez-vous et messagerie

Le patient consulte les créneaux disponibles d'un médecin, sélectionne un motif de consultation et confirme sa demande. Une messagerie sécurisée, connectée en WebSocket, permet ensuite un échange en temps réel avec le médecin, avec accusé de réception et historique complet des conversations.

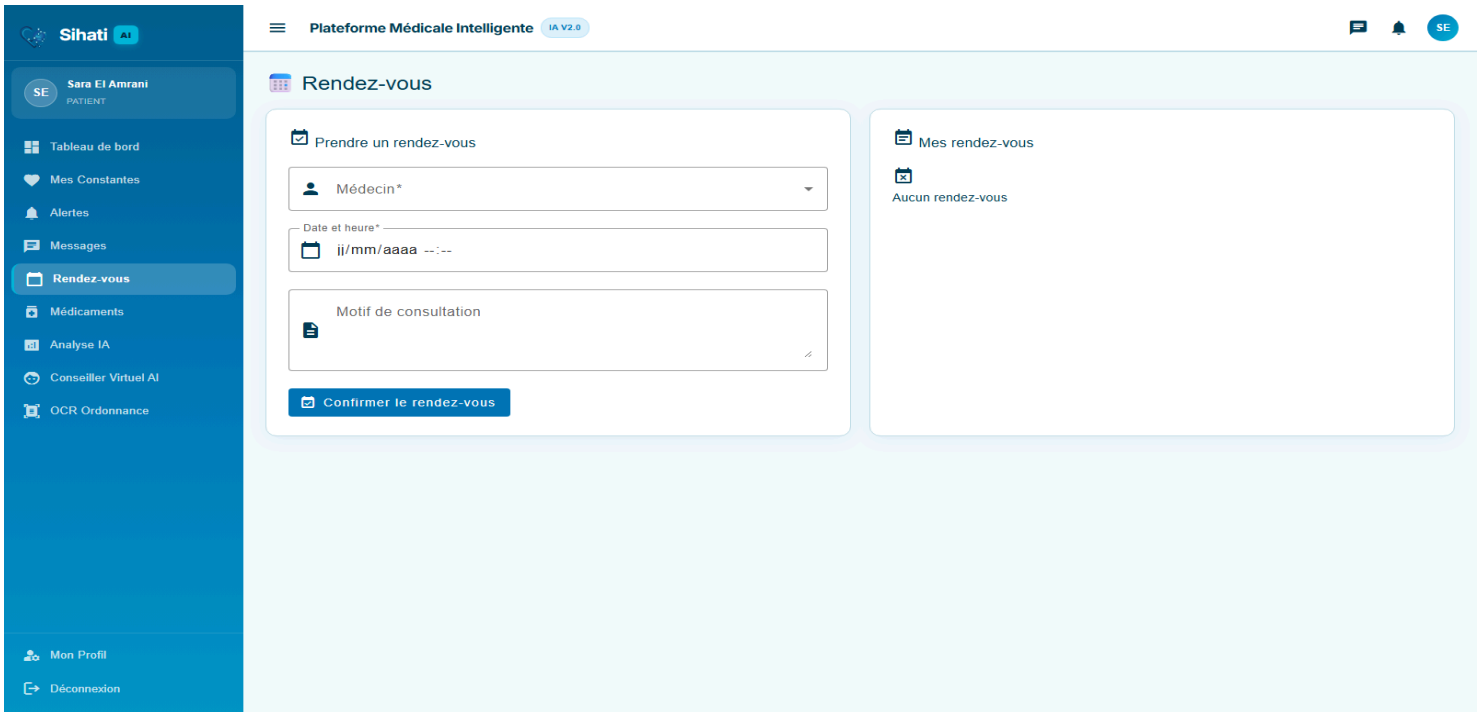


Figure 12 : Prise de rendez-vous

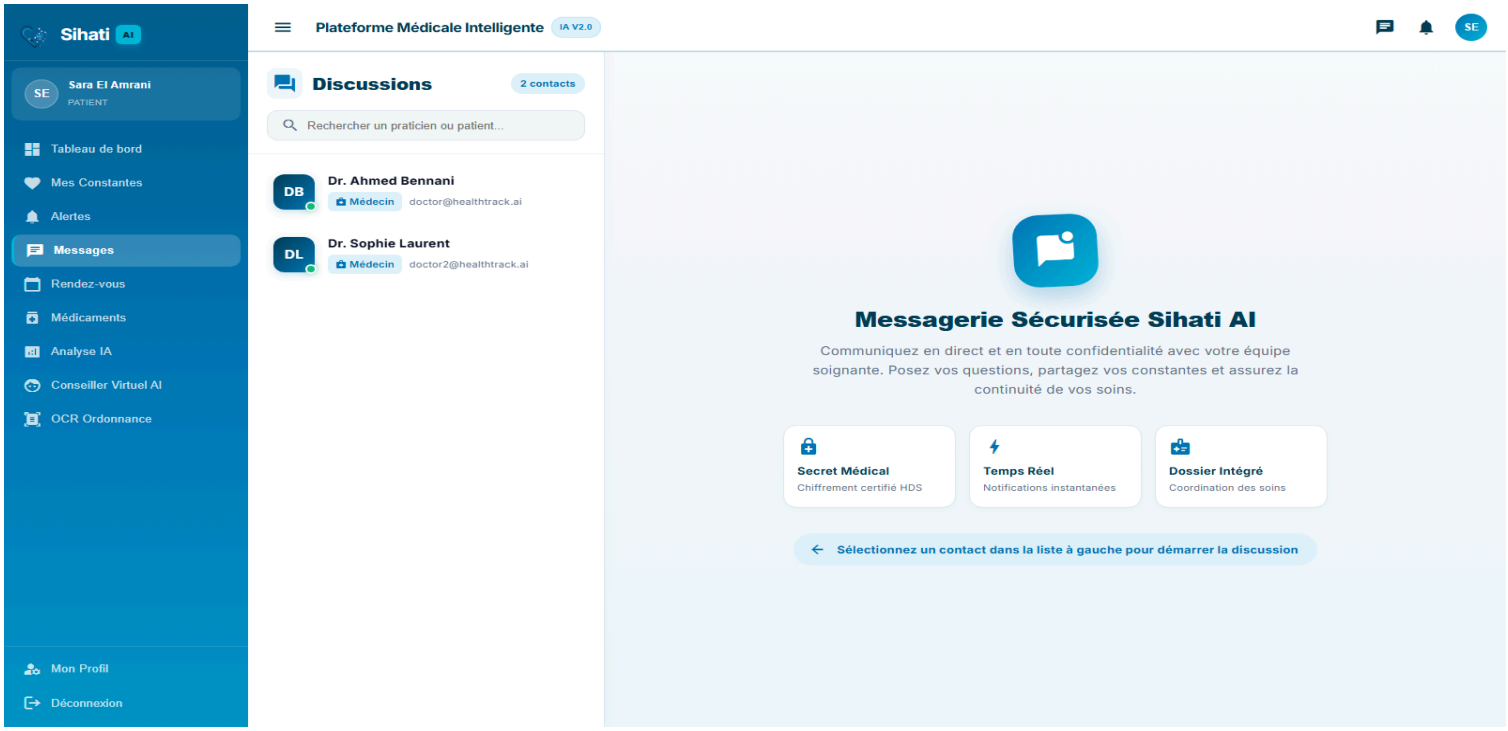


Figure 13 : Messagerie sécurisée patient-médecin

1.6. Analyse IA des tendances

En complément de la détection d'anomalie en temps réel, le patient peut lancer une analyse IA globale de ses dernières constantes vitales. Le système calcule un niveau de risque global, identifie les tendances (stable, amélioration, dégradation) et propose des recommandations de prévention personnalisées.

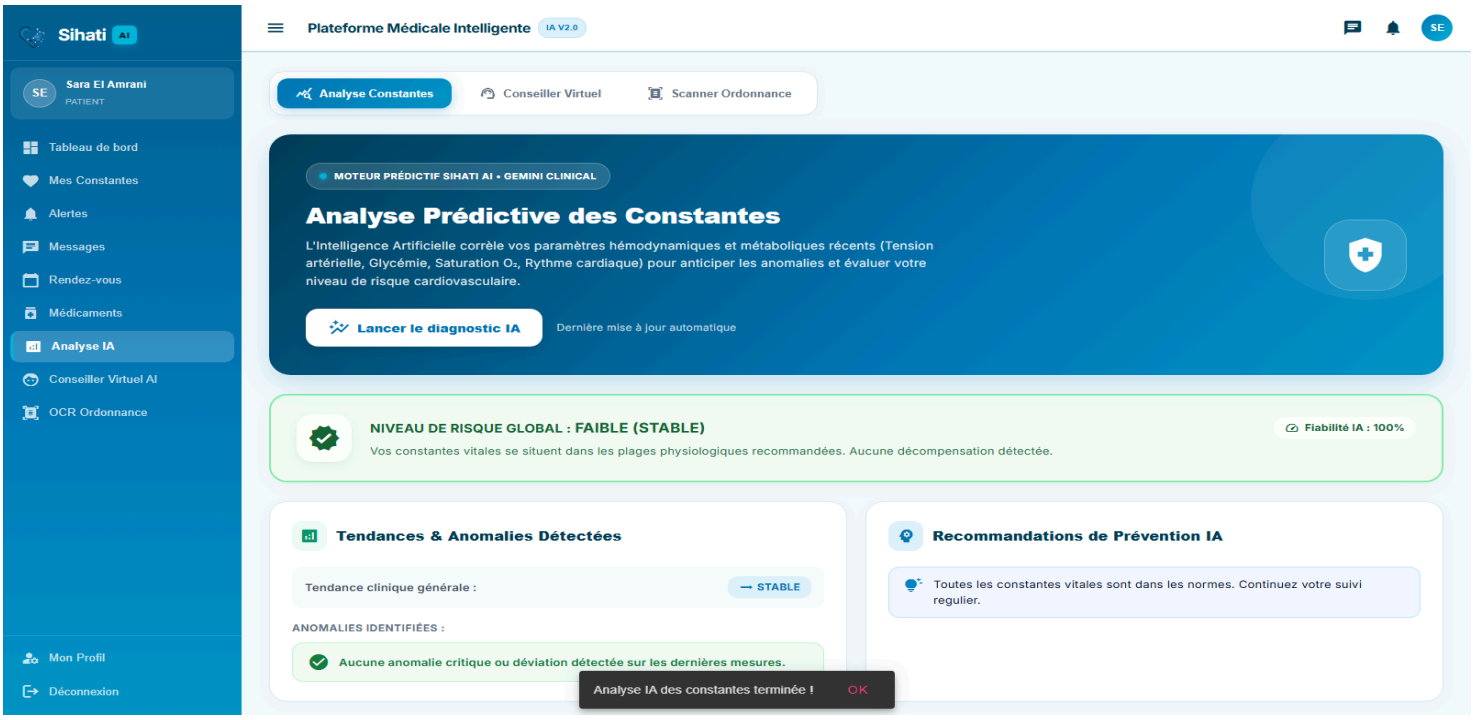


Figure 14 : Analyse IA des dernières constantes vitales

1.7. Gestion des médicaments et des alertes

Le patient consulte la liste de ses médicaments prescrits ou auto-déclarés, ainsi que l'historique de ses alertes de santé (critiques, élevées, non lues), avec la possibilité d'acquitter une alerte une fois prise en compte.

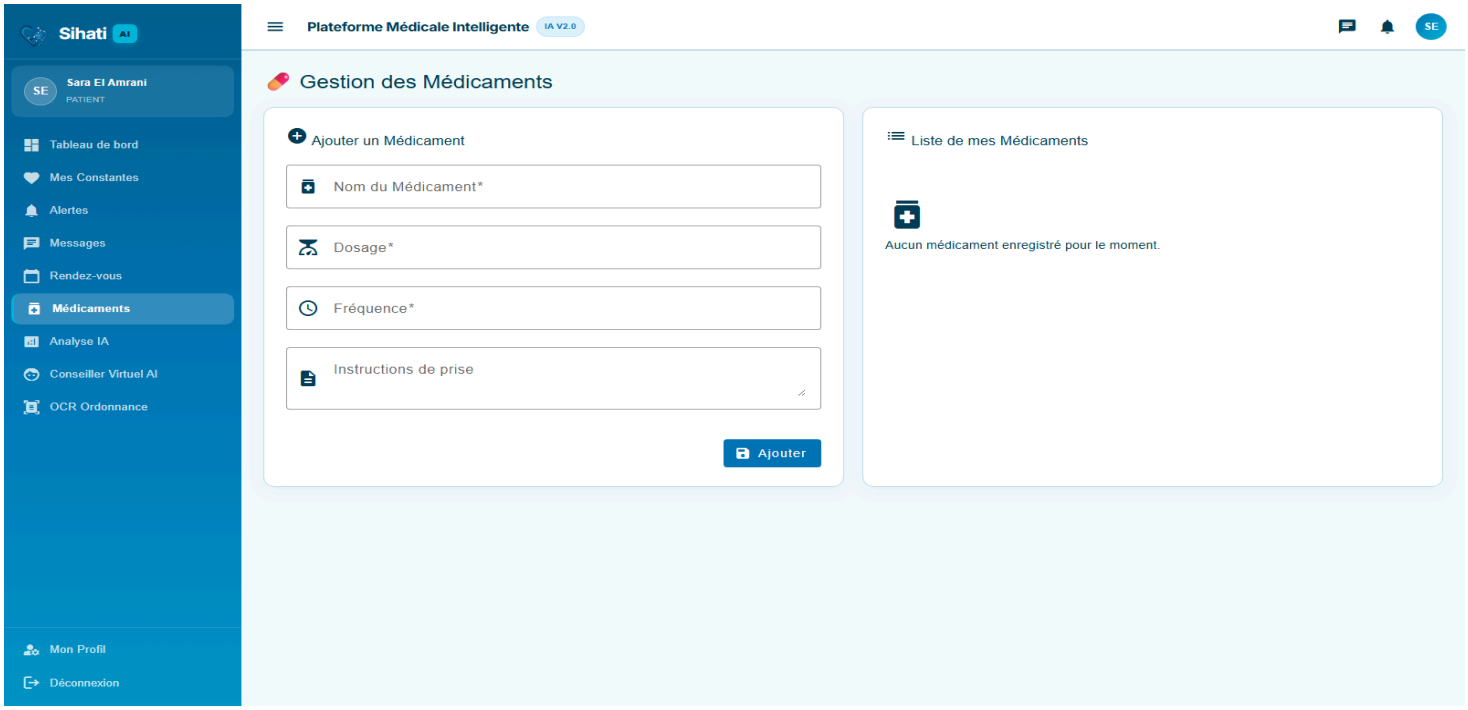


Figure 15 : Gestion des médicaments du patient

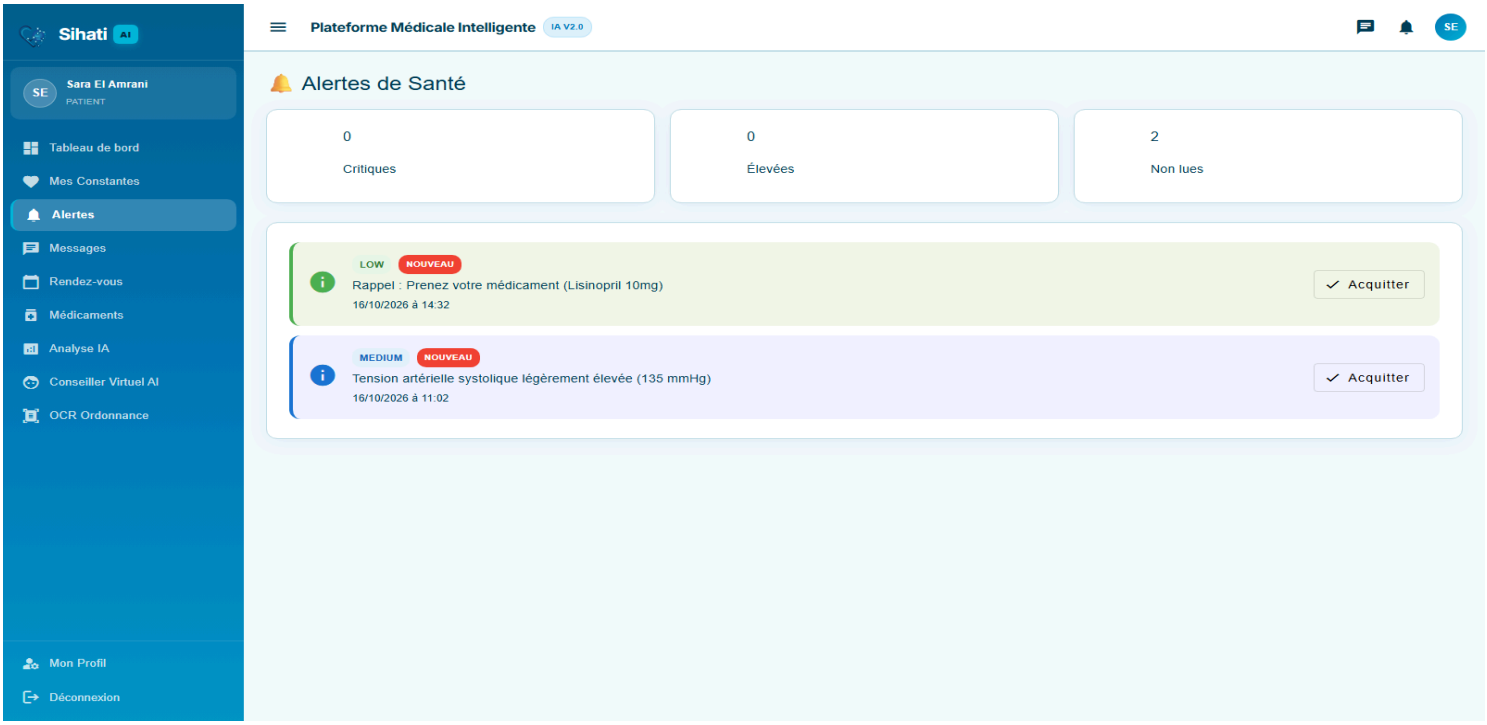


Figure 16 : Alertes de santé du patient

2. Espace Médecin (Web)

2.1. Tableau de bord médecin

Le médecin dispose d'une vue synthétique de l'ensemble de ses patients, avec mise en évidence prioritaire des alertes critiques en attente de traitement. Des graphiques d'évolution (Chart.js) permettent de visualiser rapidement la tendance des constantes d'un patient sur les dernières mesures.

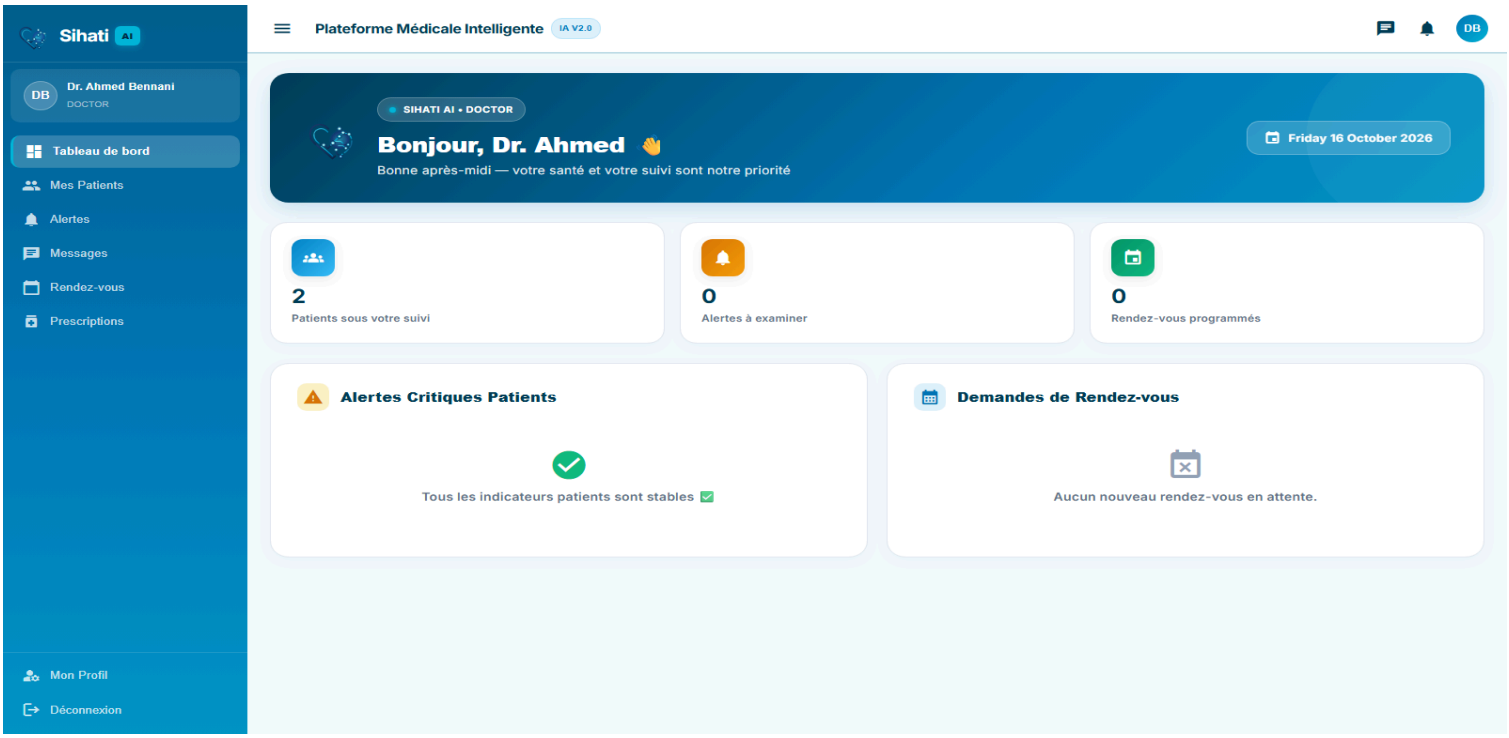


Figure 17 : Tableau de bord médecin

2.2. Gestion des prescriptions

Le médecin peut créer, modifier ou désactiver une prescription associée à un patient, en précisant le nom du médicament, le dosage et la fréquence de prise. Chaque prescription est immédiatement visible dans l'espace patient correspondant.

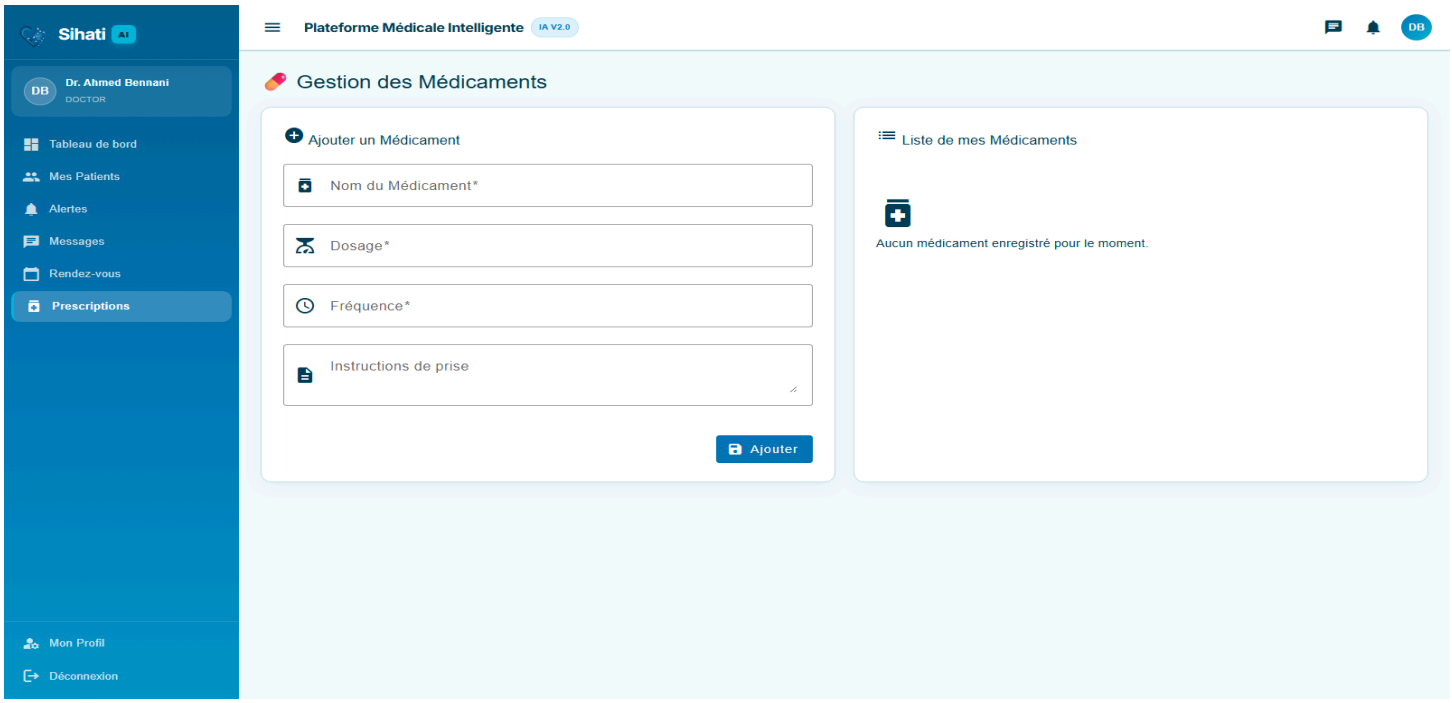


Figure 18 : Gestion des prescriptions médicales

2.3. Messagerie et suivi des alertes

Le médecin reçoit une notification en temps réel dès qu'une alerte critique est générée pour l'un de ses patients, et peut ouvrir directement le dossier concerné pour engager la conversation via la messagerie sécurisée.

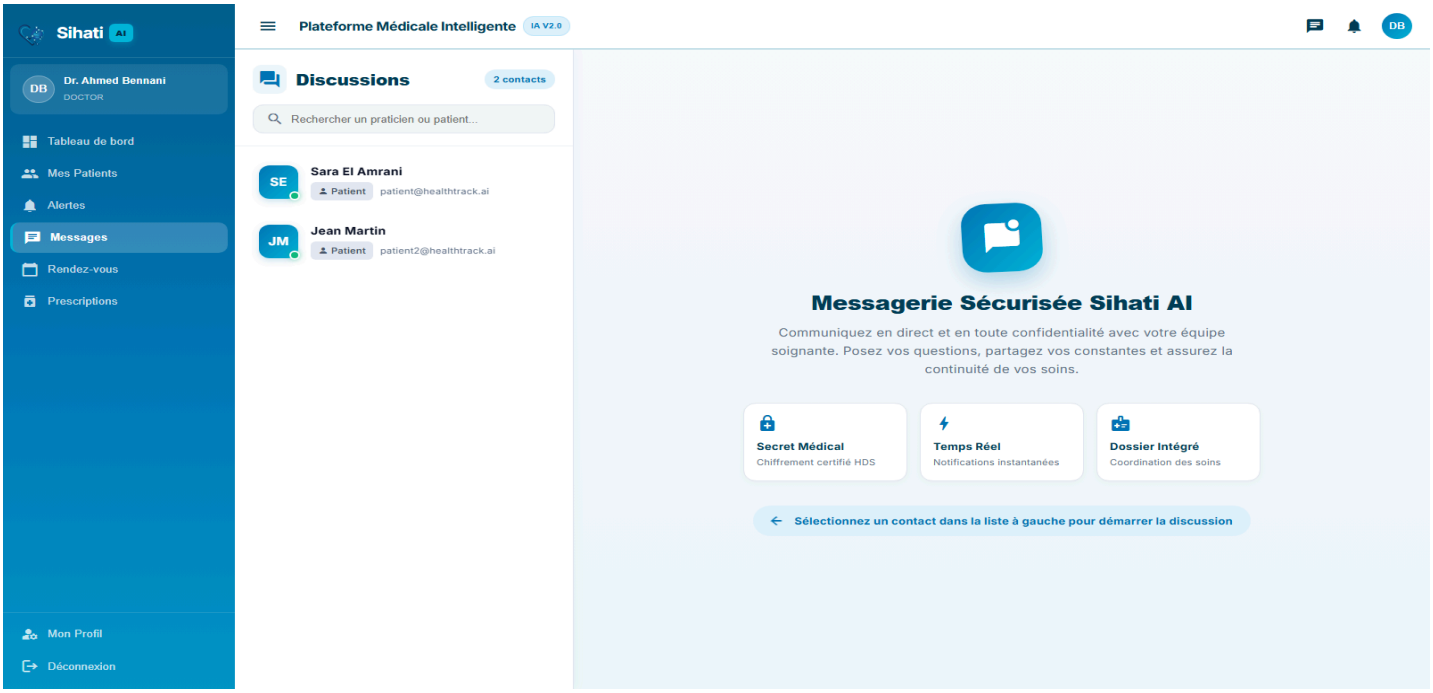


Figure 19 : Messagerie sécurisée médecin-patient

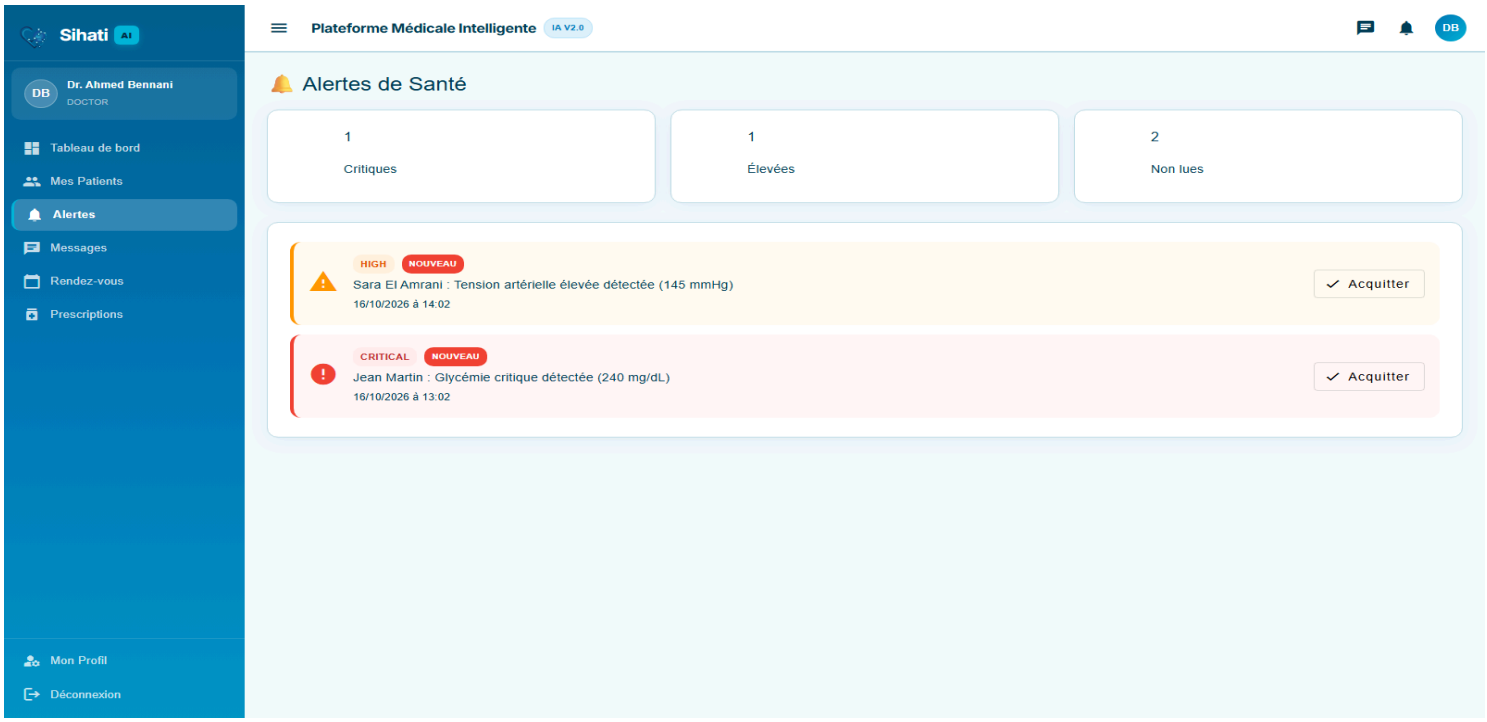


Figure 20 : Suivi des alertes de santé des patients

2.4. Gestion des patients

Le médecin dispose d'une liste consolidée de ses patients avec leurs informations clés (groupe sanguin, taille, poids) et l'historique récent de leurs constantes vitales, facilitant un suivi rapide lors d'une consultation.

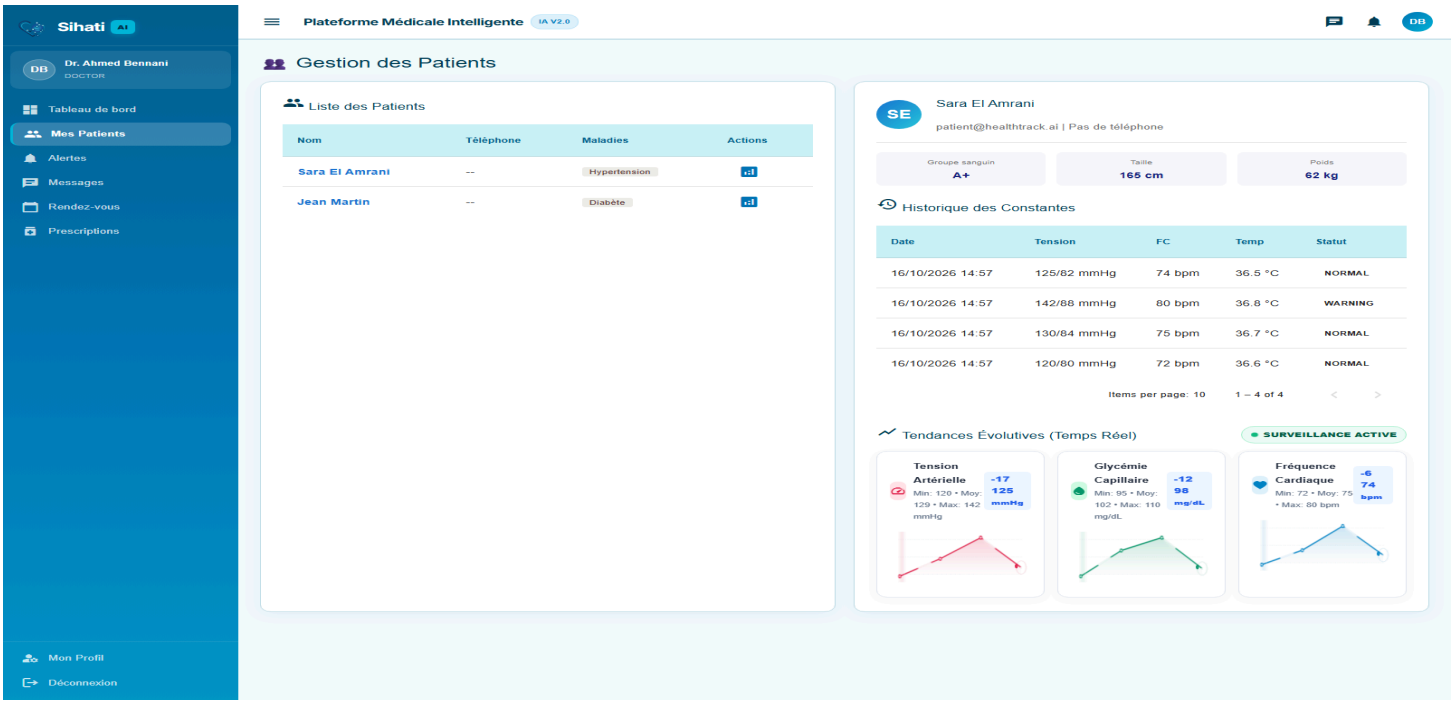


Figure 21 : Gestion des patients (vue médecin)

2.5. Prise de rendez-vous (vue médecin)

Le médecin consulte et gère ses rendez-vous à venir, confirmés ou en attente, directement depuis son espace dédié.

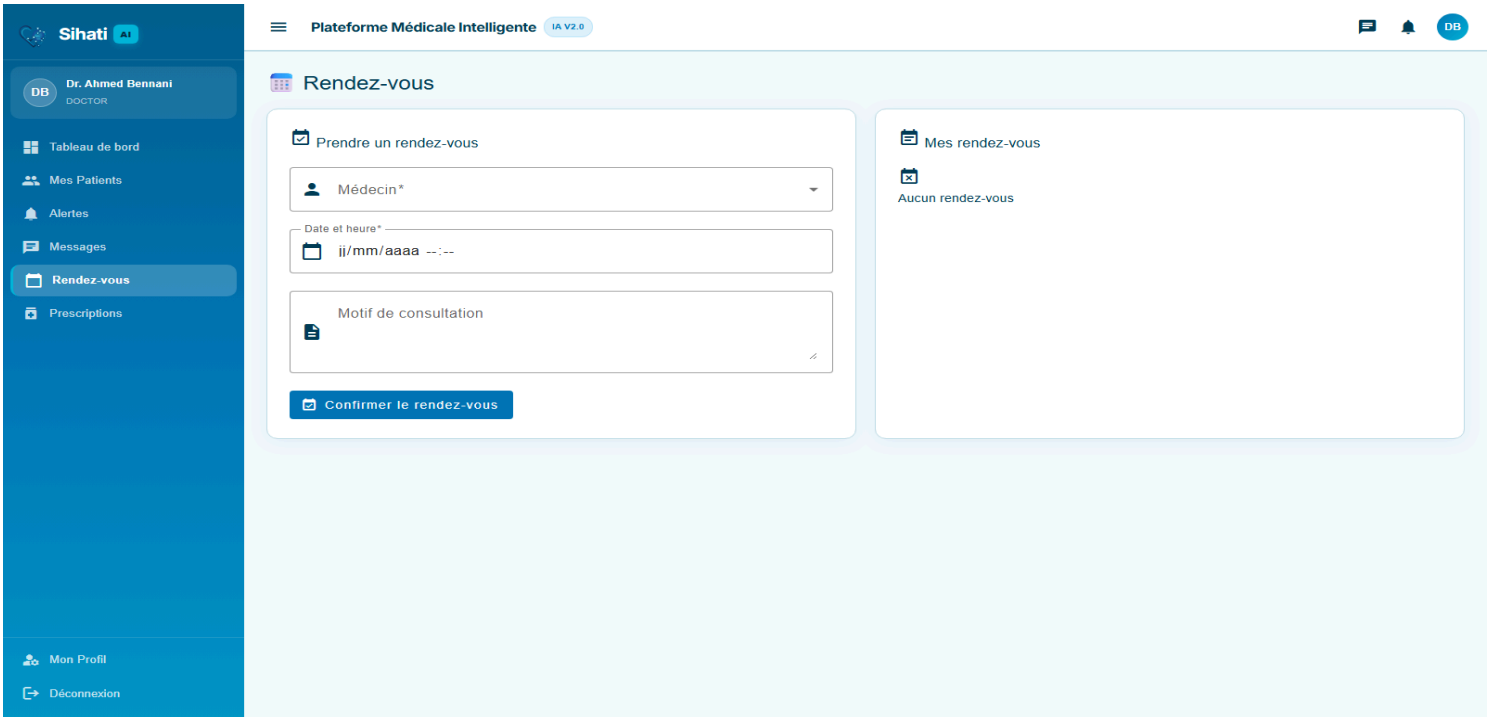


Figure 22 : Gestion des rendez-vous (vue médecin)

3. Espace Administrateur (Web)

3.1. Gestion des utilisateurs

L'administrateur gère l'ensemble des comptes de la plateforme (patients, médecins, autres administrateurs), avec attribution des rôles (RBAC) et réinitialisation des accès en cas de besoin.

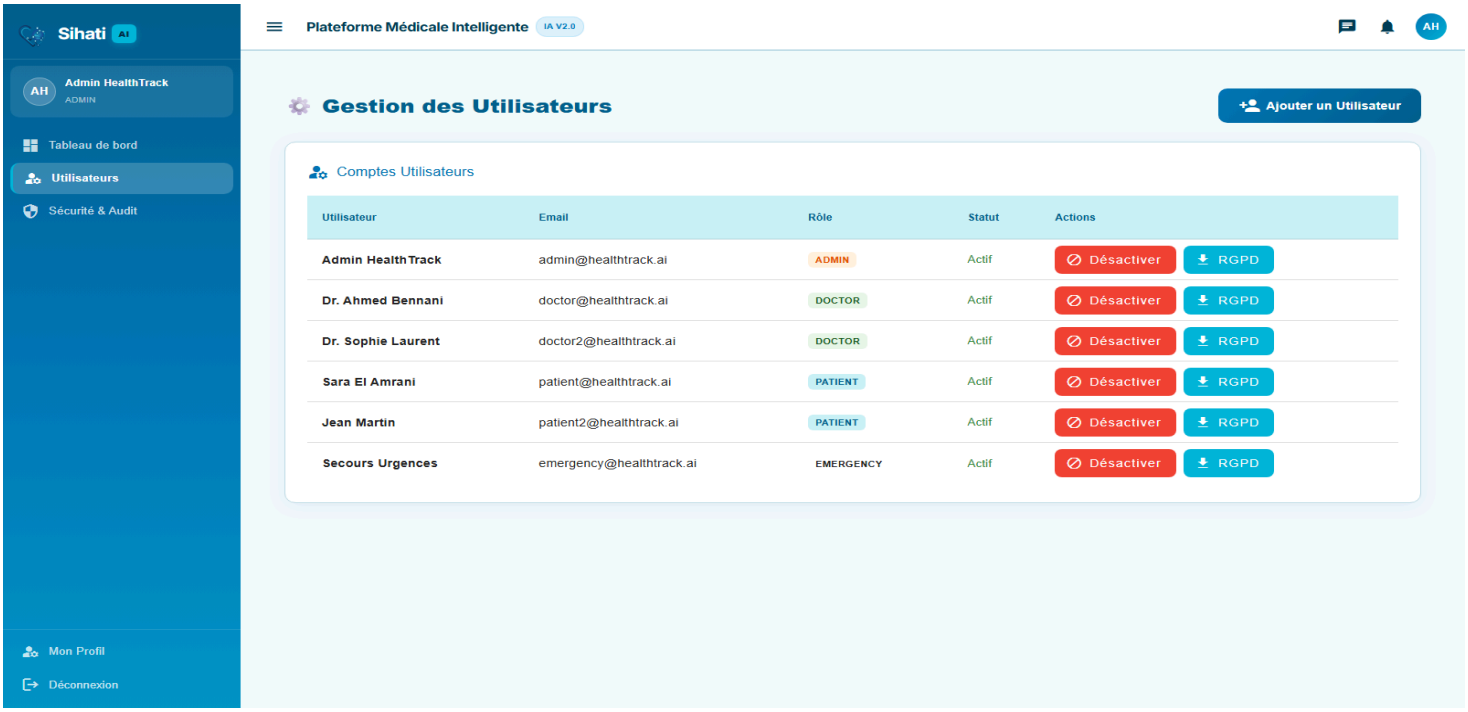


Figure 23 : Gestion des utilisateurs (vue administrateur)

3.2. Audit de sécurité

Un tableau consolidé présente l'historique des connexions, des tentatives échouées et des actions sensibles réalisées sur la plateforme (table AuditLog / LoginAttempt), conformément aux exigences de traçabilité RGPD.

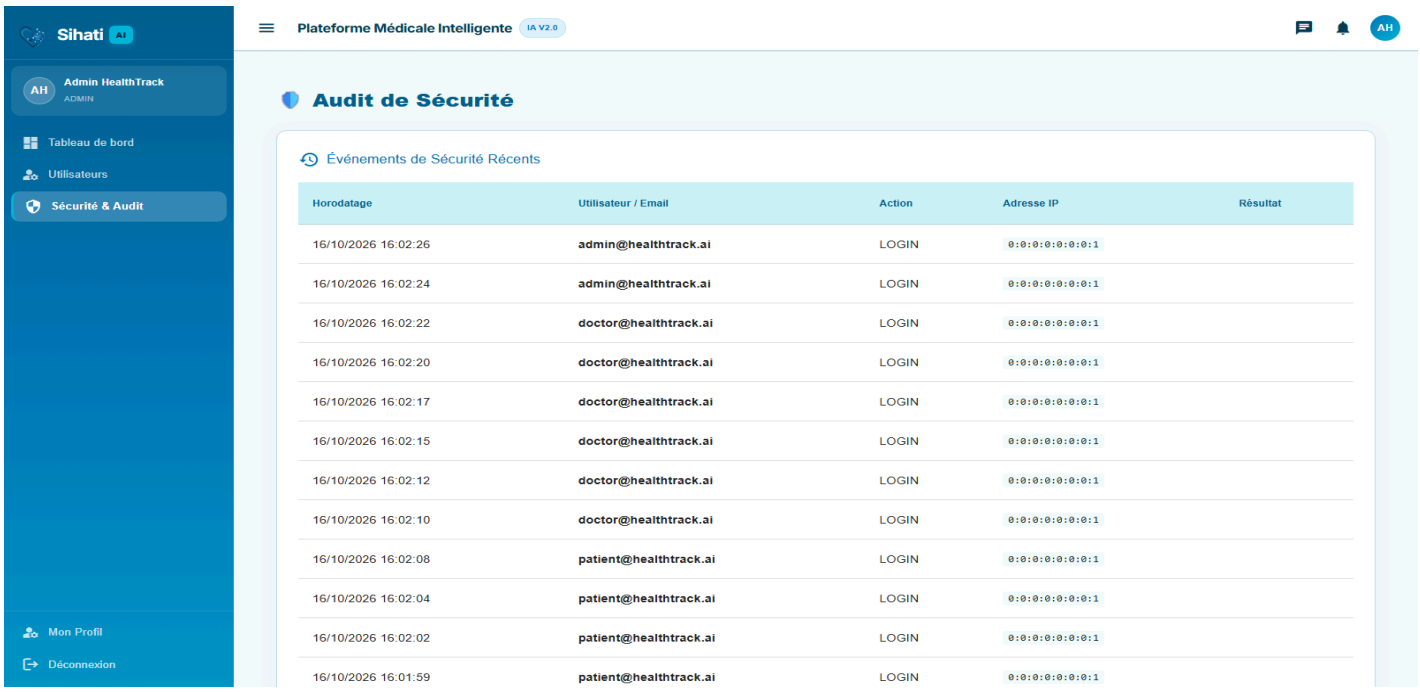


Figure 24 : Audit de sécurité — historique des connexions

3.3. Statistiques et rapports

L'administrateur dispose d'une vue synthétique des indicateurs clés de la plateforme (utilisateurs inscrits, alertes générées, mesures de santé enregistrées, rendez-vous, comptes actifs/inactifs), avec un accès rapide à la gestion des utilisateurs et à l'audit de sécurité.

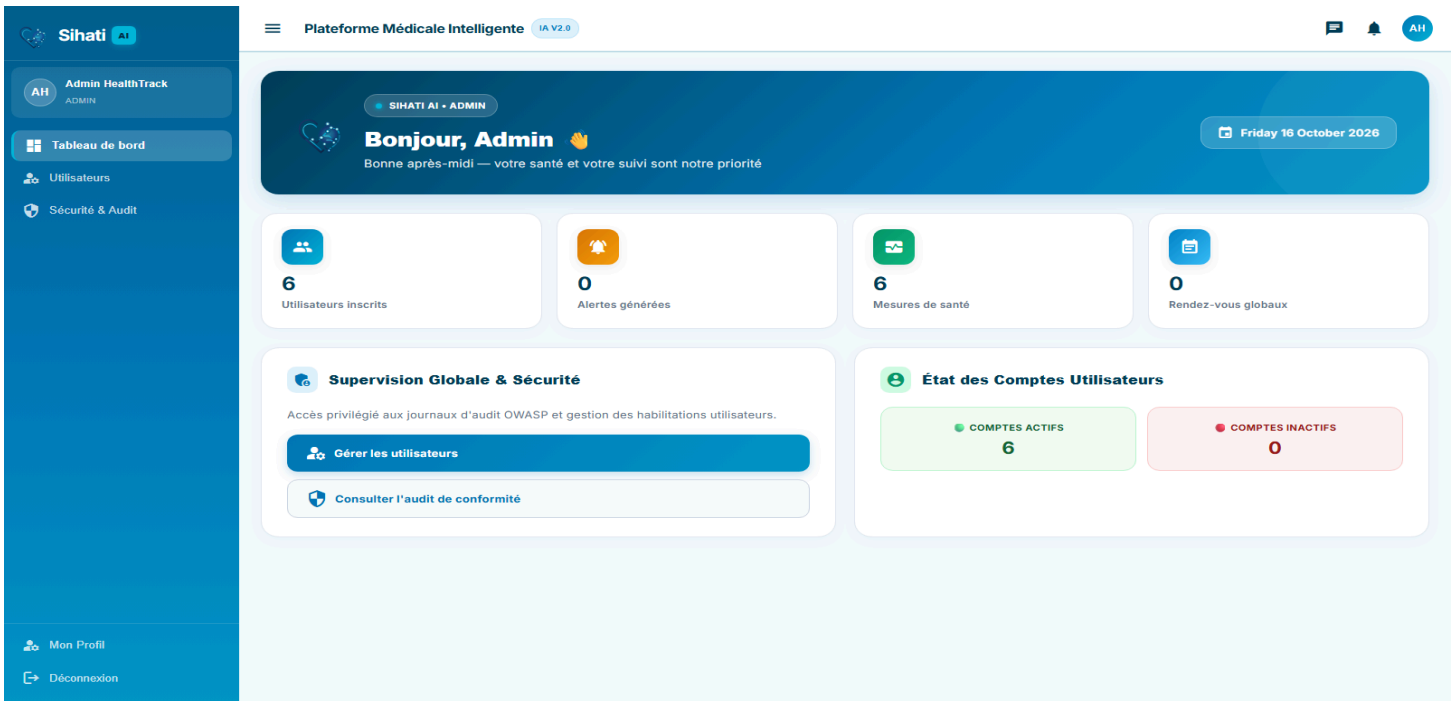


Figure 25 : Tableau de bord administrateur — indicateurs globaux

3.4. Création de comptes

L'administrateur (ou tout nouvel utilisateur lors de l'inscription) peut créer un compte en précisant son rôle : Patient, Médecin ou Urgentiste, chacun bénéficiant d'un espace et de permissions adaptés (RBAC).

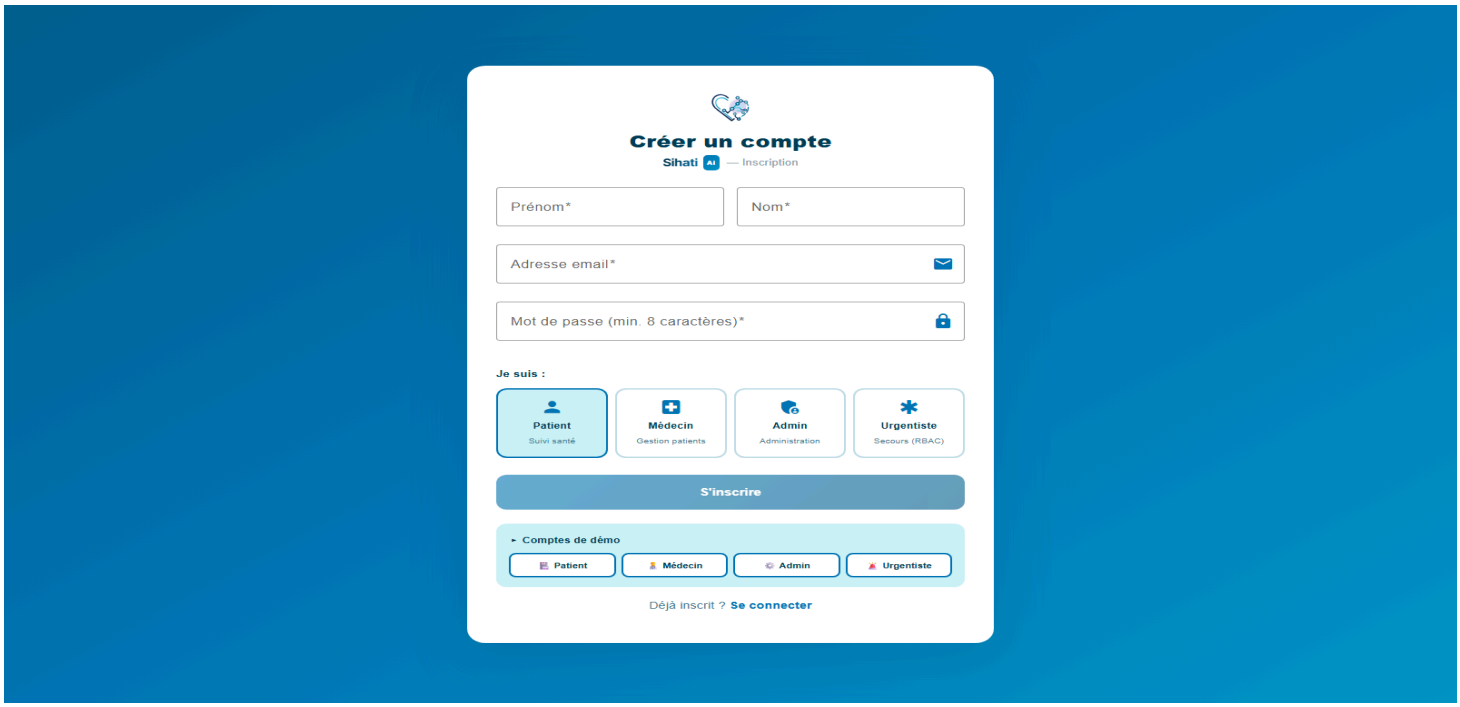


Figure 26 : Création d'un compte utilisateur

4. Espace Urgentiste (accès d'urgence)

Le rôle Urgentiste (acteur « Secours » identifié dans le diagramme de cas d'utilisation) offre un accès en lecture seule aux données critiques d'un patient en situation d'urgence : constantes vitales récentes, alertes actives, médicaments en cours et coordonnées de contact d'urgence. Cet accès élargi mais restreint (RBAC dédié) permet aux services de secours d'intervenir rapidement avec les informations médicales essentielles, sans donner accès aux fonctionnalités de gestion réservées au patient ou au médecin traitant.

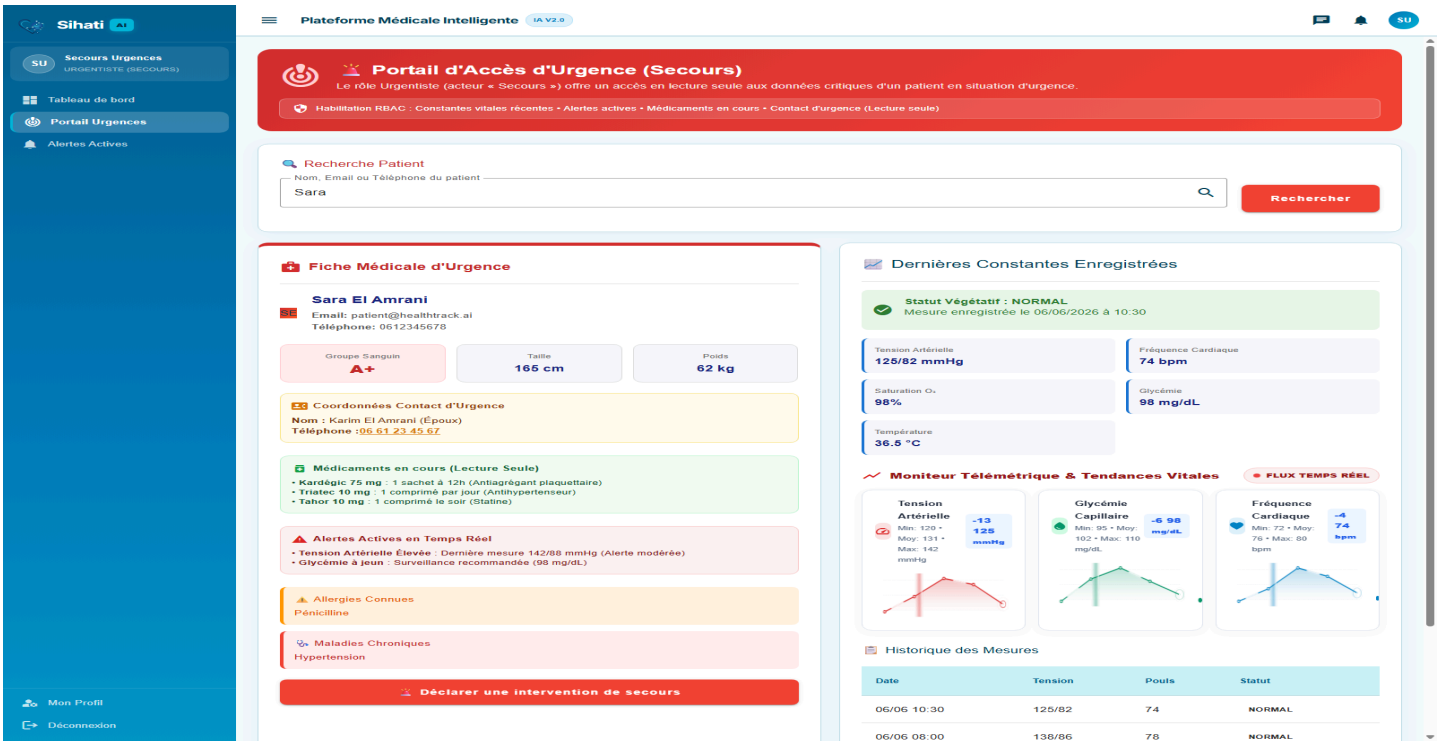


Figure 27 : Tableau de bord de l'espace Urgentiste

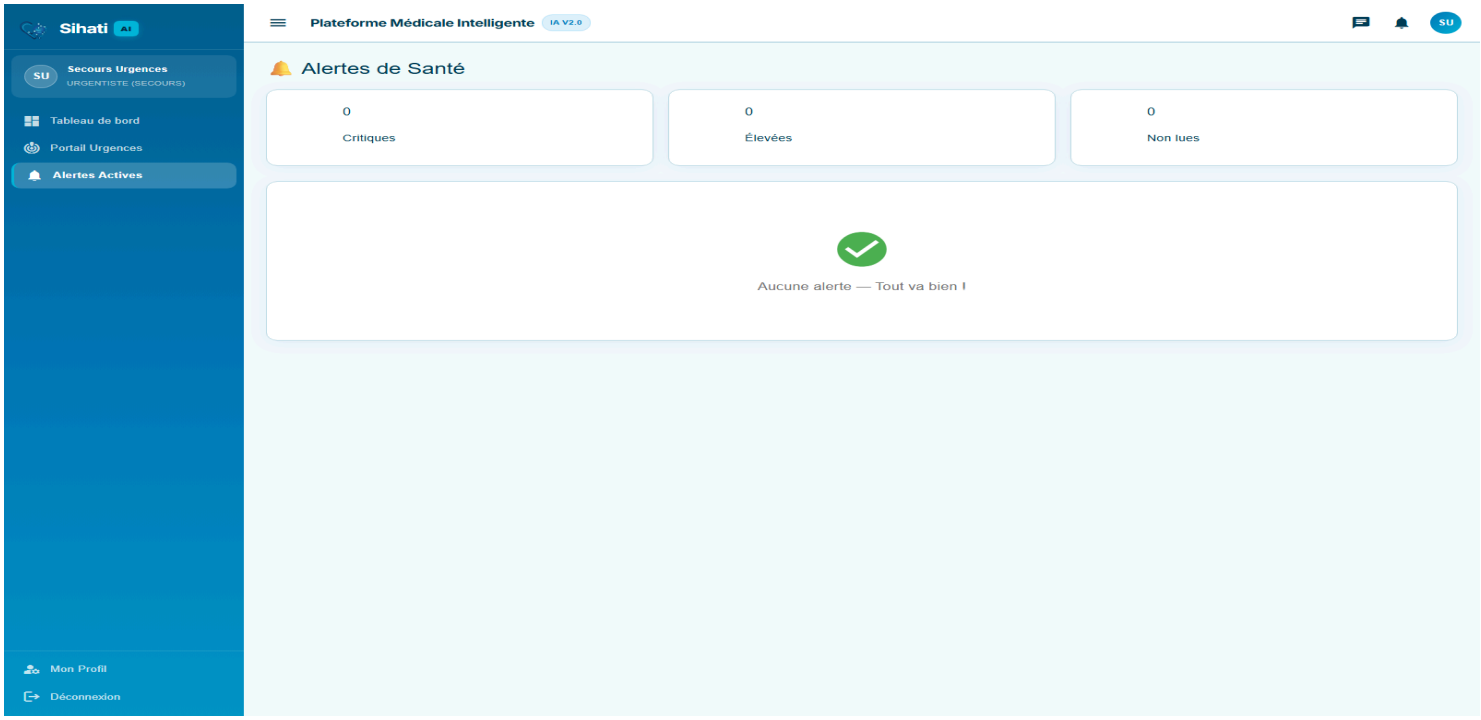


Figure 28 : Consultation des alertes de santé — accès urgence

5. Backend et API REST

Le backend expose une API REST documentée via Swagger UI, sécurisée par JWT (HMAC-SHA256, access token 24h / refresh token 7 jours) et protégée par un contrôle d'accès par rôle (@PreAuthorize) sur chaque endpoint sensible.

Méthode	Endpoint	Description
POST	/api/auth/login	Connexion et génération du JWT
POST	/api/auth/register	Inscription d'un nouvel utilisateur
GET / POST	/api/health-records	Consulter / enregistrer une constante vitale
GET	/api/alerts	Récupérer les alertes actives d'un patient
GET / POST	/api/appointments	Lister / créer un rendez-vous
POST	/api/ocr/prescription	Analyser une ordonnance (PDF/Image)
POST	/api/messages/{userId}	Envoyer un message sécurisé

6. Sécurité

- **Authentification :** JWT HMAC-SHA256, access token 24h, refresh token 7 jours, 2FA optionnelle par code OTP email (5 minutes).
- **Autorisation (RBAC) :** 4 rôles — PATIENT, DOCTOR, ADMIN, EMERGENCY — avec contrôle d'accès sur chaque endpoint.
- **Rate limiting :** blocage automatique après 5 tentatives de connexion échouées en 15 minutes.
- **Protection des données médicales :** chiffrement AES-128 des champs sensibles, conformité RGPD (audit trail, droit à l'oubli).
- **Sécurité OWASP Top 10 :** RBAC via @PreAuthorize, protection anti-injection SQL (Spring Data JPA), en-têtes HSTS/CSP/X-XSS-Protection, rate limiting, tables AuditLog et LoginAttempt.

7. Application Mobile (Android)

En complément de l'application web, Sihati AI propose une application Android native (Java, Retrofit 2.9, Material Design 3, minSdk 24 / targetSdk 34) qui donne au patient un accès complet à son suivi médical depuis son smartphone : connexion sécurisée, tableau de bord, saisie et historique des constantes, avatar IA vocal, scan OCR d'ordonnances, prise de rendez-vous, gestion des médicaments et bouton SOS d'urgence. L'interface adopte les mêmes codes visuels et les mêmes règles métier que la version web, garantissant une expérience cohérente sur tous les supports.

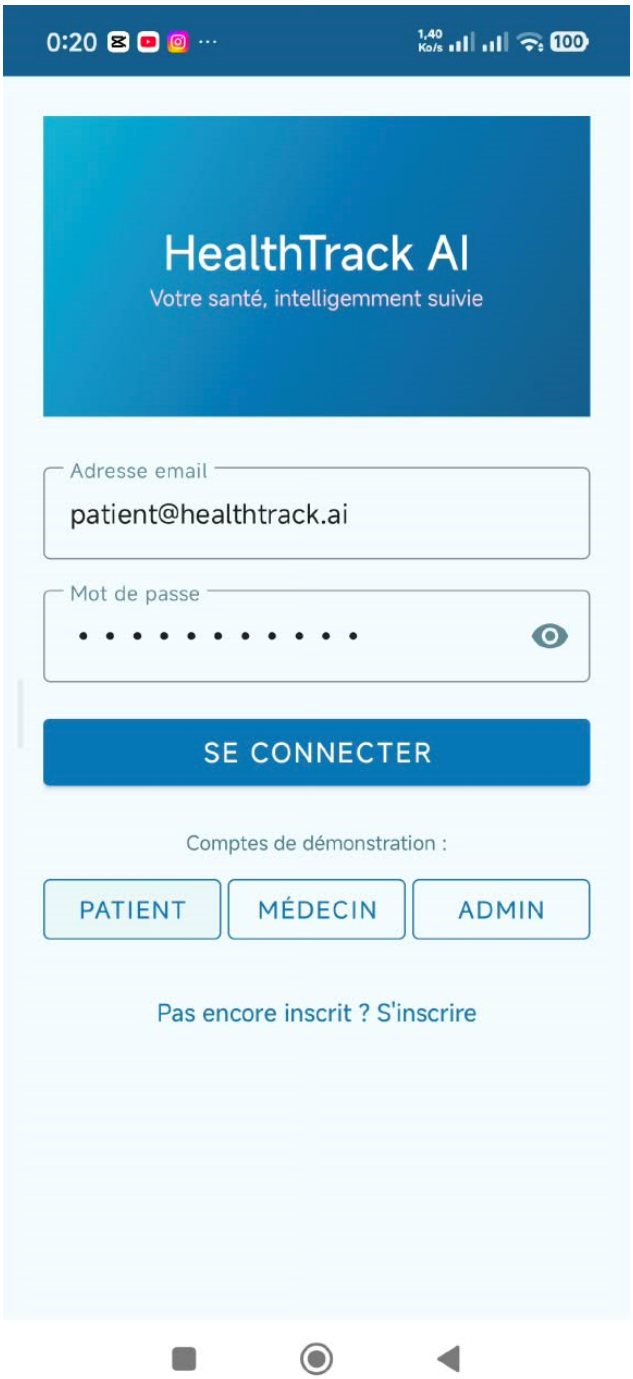


Figure 29 : Connexion (mobile)



Figure 30 : Tableau de bord patient (mobile)

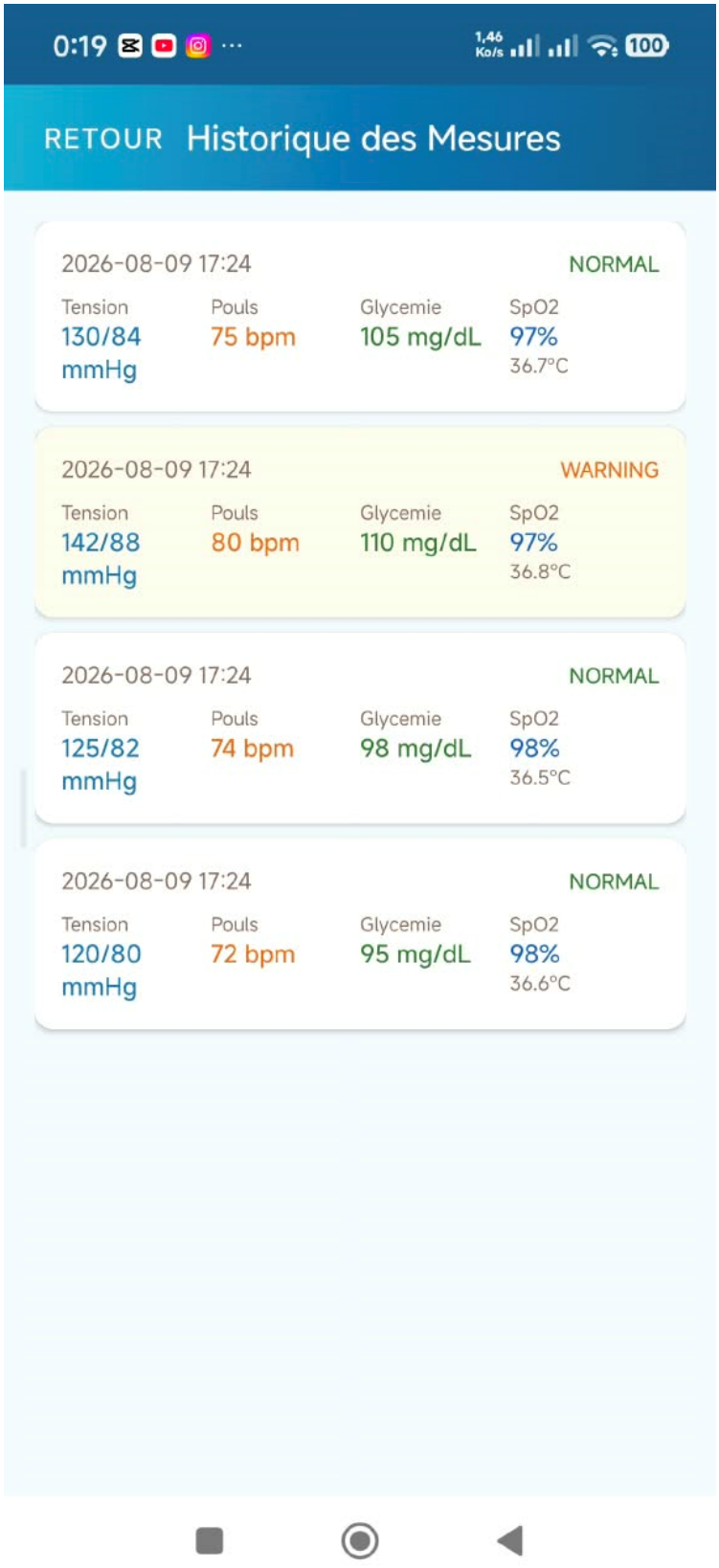


Figure 31 : Historique des mesures (mobile)

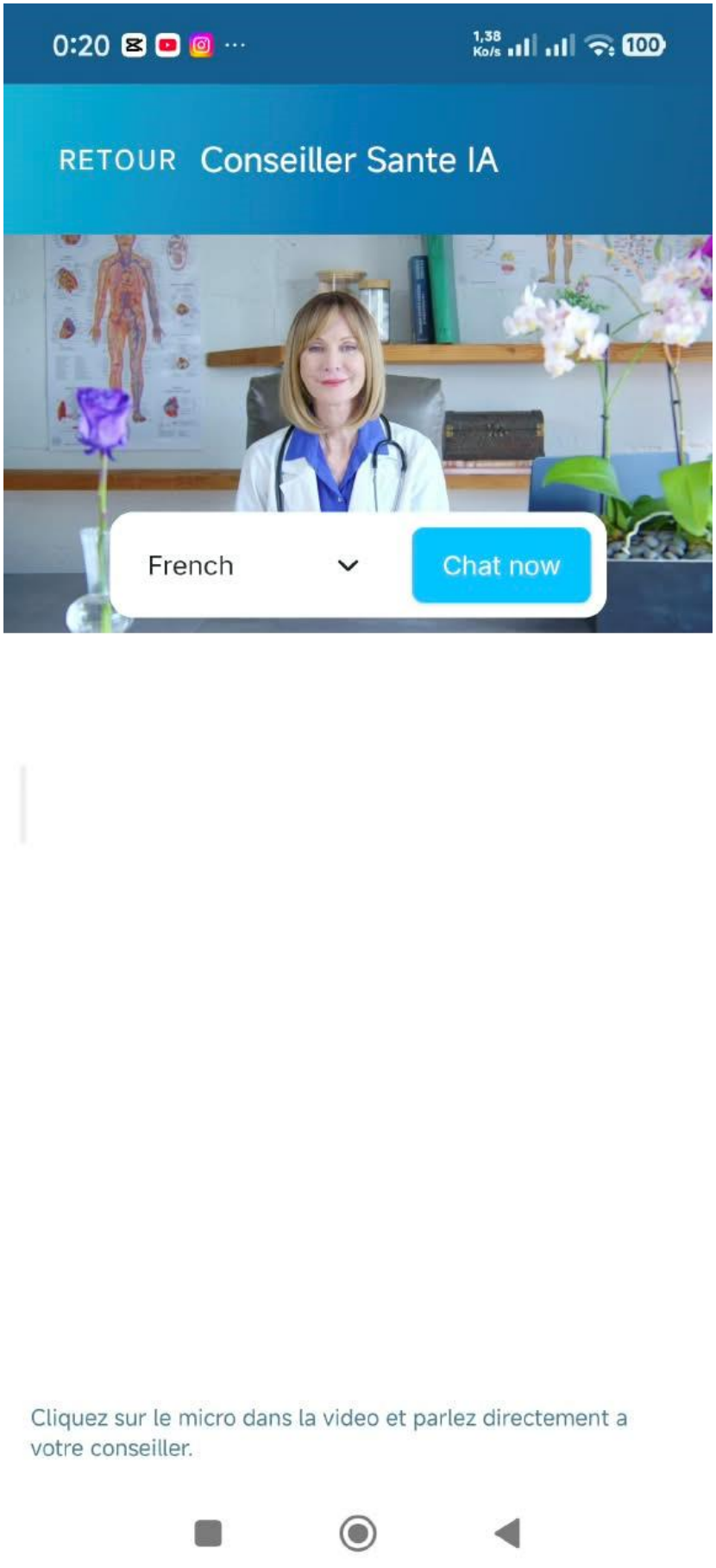


Figure 32 : Conseiller santé IA (mobile)

Le scan d'ordonnance par caméra ou galerie, la prise de rendez-vous, la gestion des médicaments et le bouton SOS sont également disponibles nativement, avec la même logique métier que côté web (extraction OCR, statuts de rendez-vous, alertes en temps réel).



Figure 33 : Scan OCR d'ordonnance (mobile)

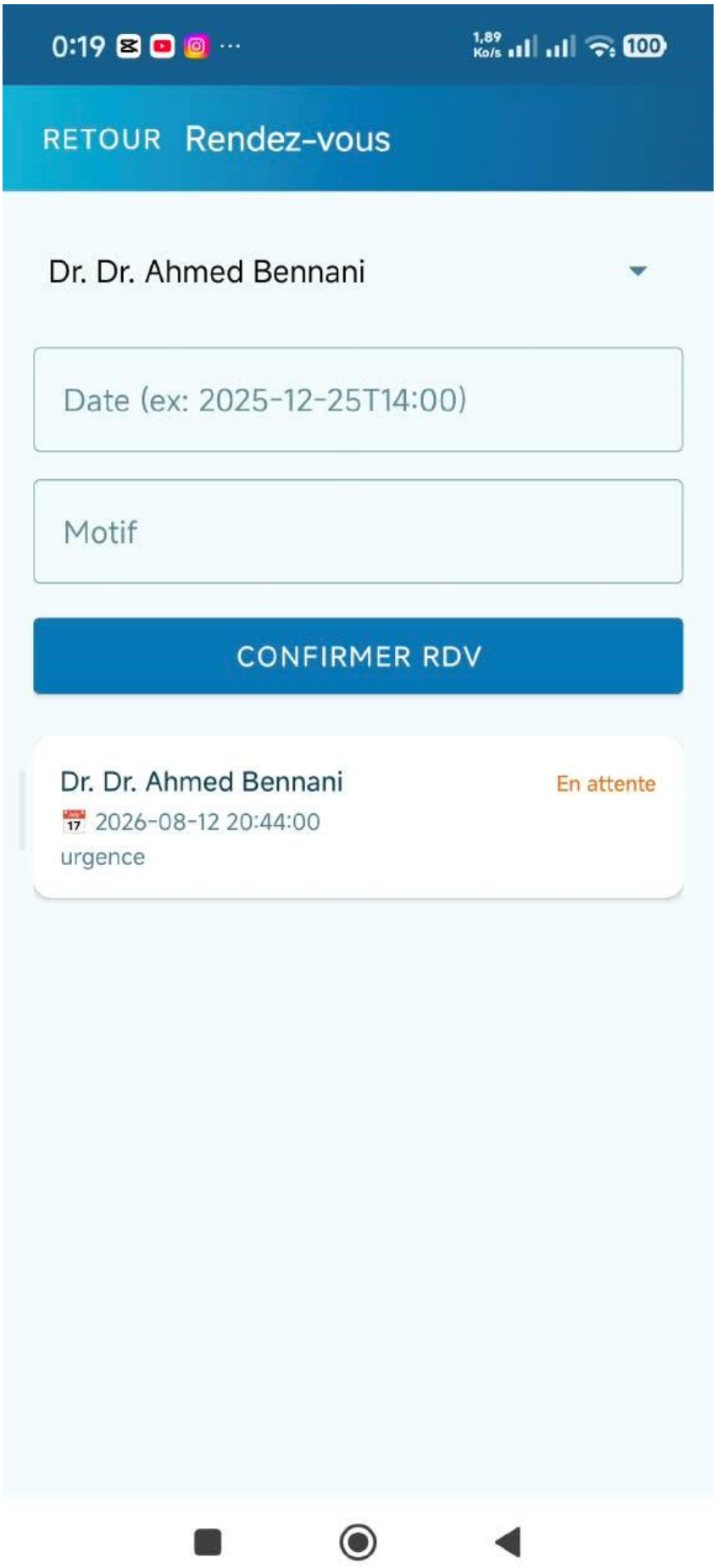


Figure 34 : Prise de rendez-vous (mobile)



Figure 35 : Gestion des médicaments (mobile)



Figure 36 : Bouton SOS d'urgence (mobile)

V. Tests & Validation

1. Stratégie de test

La validation de Sihati AI repose sur une approche à plusieurs niveaux, adaptée aux contraintes de temps d'un projet de fin d'année :

- Tests unitaires backend (JUnit 5 + Mockito) sur les services critiques : authentification, détection d'anomalies (AnomalyDetectionService), gestion des rendez-vous et des alertes.
- Tests d'API manuels via Swagger UI et Postman pour vérifier chaque endpoint REST (codes de retour, structure des réponses, contrôle d'accès par rôle).
- Tests fonctionnels manuels sur l'interface Angular (parcours patient, médecin, administrateur) et sur l'application Android (connexion, constantes, OCR, SOS).
- Tests de sécurité ciblés : tentative d'accès à une ressource protégée sans jeton, dépassement du seuil de tentatives de connexion (RG4), accès inter-rôles refusé (RBAC).

2. Cas de test représentatifs

Le tableau suivant présente un extrait représentatif des cas de test exécutés, couvrant les règles de gestion critiques définies au chapitre I.

ID	Scénario	Donnée d'entrée	Résultat attendu	Statut
TC01	Connexion avec identifiants valides	Email + mot de passe corrects	Redirection vers le tableau de bord, token JWT généré	Réussi
TC02	5 échecs de connexion en 15 min	Mot de passe erroné × 5	Compte bloqué temporairement (RG4)	Réussi
TC03	Tension systolique élevée	Tension = 190/100 mmHg	Statut CRITICAL + alerte médecin (RG1)	Réussi
TC04	Glycémie basse	Glycémie = 65 mg/dL	Statut CRITICAL déclenché (RG2)	Réussi
TC05	Saturation en oxygène basse	SpO2 = 87 %	Statut CRITICAL déclenché (RG3)	Réussi
TC06	Scan OCR d'une ordonnance lisible	Photo d'ordonnance imprimée	Médicaments et posologies extraits automatiquement	Réussi
TC07	Prise de RDV sur créneau indisponible	Créneau déjà réservé	Refus avec message d'erreur (RG8)	Réussi
TC08	Émission de prescription par un patient	Rôle = PATIENT	Accès refusé (RG5, RBAC)	Réussi
TC09	Déclenchement du bouton SOS	Clic sur SOS (web ou mobile)	Alerte immédiate envoyée au médecin	Réussi
TC10	Accès à l'audit de sécurité par un patient	Rôle = PATIENT sur /admin/audit	Accès refusé (403)	Réussi

3. Synthèse des résultats

L'ensemble des cas de test exécutés a validé le comportement attendu des règles de gestion critiques (seuils d'alerte, contrôle d'accès par rôle, verrouillage de compte). Les tests unitaires couvrent en priorité les services à forte valeur métier (authentification, détection d'anomalies) plutôt que l'ensemble du code, faute de temps pour une couverture exhaustive dans le cadre de ce projet académique. Aucun test de charge réel n'a été mené : les exigences de performance (BNF1 à BNF3) restent donc validées uniquement au niveau de la conception, et non mesurées en conditions réelles — un point détaillé dans le bilan des objectifs (chapitre VII).

VI. Gestion des risques

La conduite du projet a fait l'objet d'une identification anticipée des risques susceptibles d'affecter le planning, la qualité ou la sécurité de la plateforme. Chaque risque a été positionné selon sa probabilité d'occurrence et son impact potentiel, puis associé à une stratégie de mitigation.

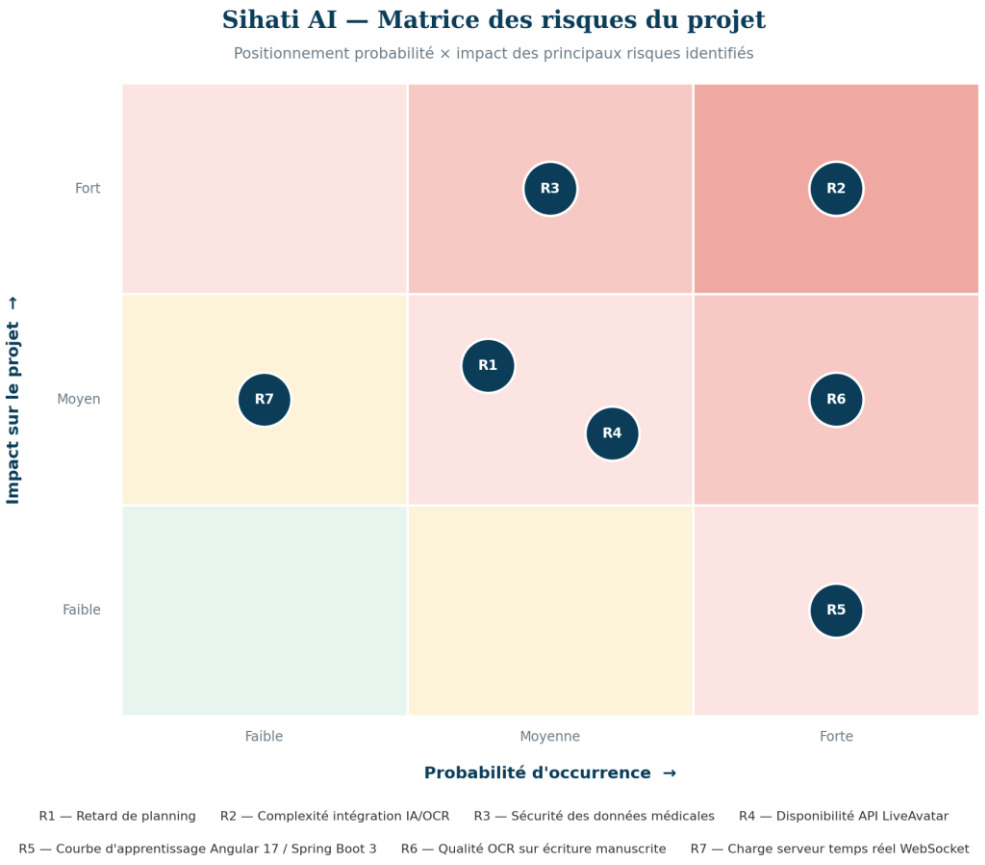


Figure 43 : Matrice des risques du projet

Risque	Probabilité	Impact	Stratégie de mitigation
R1 — Retard de planning	Moyenne	Moyen	Sprints courts, revues hebdomadaires avec l'encadrant, marge intégrée en fin de planning.
R2 — Complexité d'intégration IA/OCR	Forte	Fort	Sprint dédié (P5) et prototypage anticipé des modules IA ; solution de repli en saisie manuelle.
R3 — Sécurité des données médicales	Moyenne	Fort	Chiffrement AES-128, audit trail systématique, tests ciblés de contrôle d'accès (RBAC).
R4 — Disponibilité de l'API LiveAvatar (tiers)	Moyenne	Moyen	Gestion d'erreur gracieuse côté frontend, message de repli si le service est indisponible.
R5 — Courbe d'apprentissage (Angular 17 / Spring Boot 3)	Forte	Faible	Montée en compétences dès le Sprint 1, documentation officielle, entraide au sein de l'équipe.
R6 — Qualité OCR sur ordonnances manuscrites	Forte	Moyen	Validation manuelle des champs extraits avant import dans le dossier médicament.
R7 — Charge serveur en temps réel (WebSocket)	Faible	Moyen	Architecture STOMP/SockJS conçue pour la montée en charge ; tests de charge non prioritaires à ce stade.

VII. Bilan des objectifs atteints

Ce chapitre confronte le cahier des charges initial (chapitre I) à l'état réel de la plateforme livrée, afin de dresser un bilan objectif du projet.

1. Besoins fonctionnels

N°	Besoin fonctionnel	Statut	Référence
BF1	Gestion du dossier médical (antécédents, allergies, maladies chroniques)	Réalisé	§IV.1.1
BF2	Enregistrement des constantes vitales	Réalisé	§IV.1.2
BF3	Alertes automatiques NORMAL / WARNING / CRITICAL	Réalisé	§IV.1.2, §IV.6
BF4	Avatar IA vocal (LiveAvatar)	Réalisé	§IV.1.3
BF5	Scan d'ordonnances avec extraction OCR	Réalisé	§IV.1.4
BF6	Prise de rendez-vous en ligne	Réalisé	§IV.1.5
BF7	Messagerie sécurisée en temps réel	Réalisé	§IV.1.5
BF8	Bouton SOS d'urgence	Réalisé	§IV.1.7, §IV.7
BF9	Tableau de bord médecin multi-patients	Réalisé	§IV.2.1
BF10	Alertes critiques en temps réel (médecin)	Réalisé	§IV.2.3
BF11	Historique et graphiques d'évolution	Réalisé	§IV.2.4
BF12	Messagerie sécurisée médecin-patient	Réalisé	§IV.2.3
BF13	Gestion et émission de prescriptions	Réalisé	§IV.2.2
BF14	Gestion globale des utilisateurs et des rôles	Réalisé	§IV.3.1
BF15	Audit de sécurité complet	Réalisé	§IV.3.2
BF16	Génération de statistiques et rapports	Réalisé	§IV.3.3

Les 16 besoins fonctionnels du cahier des charges ont été implémentés et sont illustrés par des captures d'écran dans le chapitre IV.

2. Besoins non fonctionnels

N°	Besoin non fonctionnel	Statut
BNF1	Temps de réponse API < 200 ms (95% des requêtes)	Conçu pour, non mesuré en charge réelle
BNF2	Support de plusieurs centaines d'utilisateurs simultanés	Non testé (hors périmètre du PFE)
BNF3	Disponibilité du service > 99,5%	Non mesuré (pas d'environnement de production réel)
BNF4	Authentification JWT + 2FA (OTP email)	Réalisé
BNF5	Chiffrement AES-128 des données sensibles	Réalisé
BNF6	Conformité RGPD / bonnes pratiques HIPAA	Réalisé (audit trail, export des données)
BNF7	Sauvegarde automatique quotidienne de la BDD	Non implémenté — perspective d'amélioration
BNF8	Interface responsive (mobile, tablette, ordinateur)	Réalisé
BNF9	Navigation intuitive, peu de clics	Réalisé (validation qualitative par l'équipe)
BNF10	Support multilingue (français, arabe, anglais)	Partiel — français uniquement à ce stade
BNF11	Support des navigateurs récents (Chrome, Firefox, Safari, Edge)	Réalisé — testé sur Chrome, Edge, Firefox
BNF12	Export des données en PDF et Excel	Partiel — export PDF réalisé, export Excel non implémenté
BNF13	Compatibilité Android (minSdk 24, targetSdk 34)	Réalisé

Le bilan des besoins non fonctionnels est volontairement nuancé : les exigences de performance à grande échelle (BNF1-BNF3), la sauvegarde automatique (BNF7), le support multilingue complet (BNF10) et

l'export Excel (BNF12) restent des axes d'amélioration identifiés plutôt que des objectifs pleinement atteints, cohérents avec les perspectives évoquées en conclusion.

Conclusion

Le projet Sihati AI nous a permis de concevoir et de développer une plateforme intelligente complète de suivi médical, en répondant à des besoins réels liés à la sécurité des données de santé, à la réactivité face aux situations à risque et à l'amélioration de la communication entre patients et professionnels de santé.

Tout au long de ce projet, nous avons pu appliquer les connaissances acquises durant notre cycle d'ingénierie en systèmes informatiques, notamment en analyse des besoins, modélisation UML et Merise, conception de bases de données, développement web avec Angular, développement backend avec Spring Boot, ainsi que l'intégration de technologies avancées telles que la détection d'anomalies par intelligence artificielle, un avatar conversationnel vocal et la reconnaissance optique de caractères (OCR).

La mise en place de modules dédiés aux patients, médecins et administrateurs a permis de centraliser et de simplifier la gestion des dossiers médicaux, des constantes vitales, des alertes, des rendez-vous et des prescriptions. Le choix d'une architecture sécurisée (JWT, chiffrement AES, RBAC, audit) et d'une interface moderne, responsive et intuitive contribue à une meilleure expérience utilisateur et facilite l'adoption de la plateforme par les patients comme par les professionnels de santé.

Ce projet nous a aussi donné l'occasion de développer nos compétences en travail d'équipe, en gestion de projet et en résolution de problèmes concrets, tout en respectant les contraintes techniques, réglementaires et sécuritaires propres au domaine médical.

En conclusion, Sihati AI représente une solution pratique, sécurisée et innovante qui répond aux enjeux du suivi médical numérique, tout en illustrant notre capacité à transformer un cahier des charges en une plateforme pleinement opérationnelle. Ce travail constitue une première étape vers le développement de systèmes de santé numérique plus avancés.

Pour aller plus loin, ce projet pourrait être enrichi par l'intégration d'objets connectés médicaux (IoT) pour la collecte automatique des constantes, l'ajout de modèles prédictifs plus avancés pour anticiper certaines complications de santé, ainsi que le déploiement d'une version iOS de l'application mobile. Ces perspectives d'amélioration ouvrent la voie à une solution encore plus complète et adaptée aux besoins évolutifs du secteur de la santé numérique.

Références

- Spring Boot Reference Documentation — https://docs.spring.io/spring-boot
- Angular Official Documentation — https://angular.dev
- MySQL Official Documentation
- Tesseract OCR Project — https://github.com/tesseract-ocr/tesseract
- Apache PDFBox Documentation — https://pdfbox.apache.org
- OWASP Top 10 — https://owasp.org/www-project-top-ten/
- RGPD — Règlement (UE) 2016/679 du Parlement européen
- JWT.io — Introduction to JSON Web Tokens